

CSD_01 — Algorithm Analysis and Design

Paper-I, Unit 2 (second half) — the degree-only depth that sits on top of the shared data-structures notes.

Official syllabus wording for this unit:

"Primitive data types, array, stack, queue, link list, trees, sorting and searching techniques, symbol tables, hashing etc. Analysing Algorithms, the big-Oh notation, divide and conquer & greedy method, Dynamic programming, Search and traversal techniques, NP completeness."

The bold half is this file. The plain half — arrays, stacks, queues, linked lists, trees, sorting and searching *mechanics*, hashing — lives in **CORE_08_DataStructures.md** in **Materials\Shared**. **Do not re-learn it here.** This file assumes you already know how merge sort merges and how a BST inserts; what it adds is how to *analyse* those algorithms, how to *design* new ones from the four classical paradigms, and where the boundary of tractability lies.

Why this matters / where it appears

This unit is one of the two highest-yield blocks in the whole Degree elective (the other is compilers, **CSD_02**). Three reasons:

1. **It is degree-only.** Nothing in the CS (Diploma) or Computer Forensic syllabi touches asymptotic analysis, algorithm-design paradigms or NP-completeness. Every hour here is an hour that pays into exactly one paper — but it pays *heavily*.
2. **The 2025 paper hit it three times.** Section A Q6 ("Define Big-Oh"), Section B Q12 ("working principle of Quick Sort; average and worst-case complexity"), and Section C Q17 ("Describe and compare divide and conquer and greedy, with a concrete example of each" — **15 marks**). That is 30 of Paper-I's 100 marks available from this one unit.
3. **The MCQ paper's largest block is built on it.** Q1–30 of the 2025 MCQ was "programming, data structures and algorithm analysis" — 30% of a 200-mark paper. Complexity questions ("worst case of linear search", "BFS/DFS on an adjacency list", "what does it mean for a problem to be in NP", "significance of $P = NP$ ") are guaranteed, cheap and fast.

The style warning. This is *worked-example* territory, not *read-about* territory. A 15-mark answer on dynamic programming is a filled-in table with a traceback; a 15-mark answer on greedy is a built Huffman tree with the codes read off it. You cannot produce those under time pressure from having read them. **Work every table in this file by hand at least twice.** That is the whole study method for this unit.

Topic checklist

Tick a box only when you can reproduce the worked example *from a blank page*.

§	Topic	Depth	Done
1	What an algorithm is; properties; a priori vs a posteriori analysis; the RAM model	5-mark	- []
2	Asymptotic notation: O , Ω , Θ , o , ω — formal definitions and intuition	5-mark, asked 2025	- []
3	Time complexity of loops; nested, logarithmic, $\sqrt{n}$, $\log \log n$ patterns	MCQ-critical	- []
4	Recurrence relations: substitution, recursion tree, Master Theorem	10/15-mark	- []
5	Divide and conquer: binary search, merge sort, quick sort, Strassen	15-mark, asked 2025	- []

6	Greedy method: fractional knapsack, activity selection, Huffman , Prim, Kruskal, Dijkstra; when greedy is provably correct	15-mark, asked 2025	- []
7	Dynamic programming: 0/1 knapsack, LCS , matrix chain, Floyd–Warshall, optimal BST; DP vs D&C	15-mark	- []
8	Backtracking (n-queens, graph colouring, Hamiltonian cycle) and branch & bound	10-mark	- []
9	Search and traversal techniques: BFS, DFS, applications, complexity	5/10-mark	- []
10	Symbol tables: organisation, operations, implementations, trade-offs	5-mark	- []
11	NP-completeness: P, NP, NP-hard, NP-complete, reductions, Cook's theorem	MCQ-guaranteed + 10-mark	- []
12	The master comparison table of all four paradigms	Exam armour	- []

1. Analysing algorithms

Concept — in plain English first

Suppose two people write two different programs to sort a list. One runs in 3 seconds on your laptop, the other in 5. Which algorithm is better?

You cannot tell. The 3-second one might have been run on a faster machine, in a faster language, on a smaller list, with a better compiler. Timing a program measures *that run of that program on that machine*, not the algorithm underneath it.

What we actually want to know is: **as the input gets bigger, how fast does the work grow?** If doubling the input doubles the work, the algorithm scales beautifully. If doubling the input quadruples the work, it will be fine on 1,000 items and hopeless on 1,000,000. That growth *rate* is a property of the algorithm alone — it survives changing the machine, the language and the compiler. That is what algorithm analysis measures, and asymptotic notation is the vocabulary for stating it.

Key points

What an algorithm is. A finite, ordered sequence of unambiguous, effective steps that transforms an input into the required output. The five classical properties:

Property	Meaning
Finiteness	Must terminate after a finite number of steps
Definiteness	Every step is precisely and unambiguously defined
Input	Zero or more externally supplied quantities
Output	At least one quantity, bearing a specified relation to the input
Effectiveness	Every operation is basic enough to be carried out exactly, in principle by a person with pencil and paper, in finite time

Two ways to analyse:

	A priori analysis (theoretical)	A posteriori analysis (empirical)
--	--	--

When	Before/without running	After running
What is measured	Count of fundamental operations as a function of input size	Actual time and memory consumed
Depends on machine?	No	Yes
Depends on language/compiler?	No	Yes
Result	A function, e.g. $\Theta(n \log n)$	Numbers, e.g. 3.2 s / 40 MB
Also called	Performance <i>analysis</i>	Performance <i>measurement</i>

The model of computation — the RAM model. Analysis assumes a single-processor Random Access Machine with unit-cost basic operations: one arithmetic operation, one comparison, one assignment, one array access all cost 1. No concurrency, no memory hierarchy. This is a deliberate simplification — it is *why* the answer is machine-independent.

Two resources are analysed:

- **Time complexity** — count of fundamental (basic/elementary) operations as a function of input size n .
- **Space complexity** — memory as a function of n . Split into **fixed part** (code, constants, simple variables) and **variable part** (dynamic allocation, recursion stack). *Auxiliary* space is the variable part only, excluding the input itself. Merge sort is $O(n)$ auxiliary; heap sort is $O(1)$ auxiliary; recursive quick sort is $O(\log n)$ auxiliary for the stack.

Three cases:

Case	Meaning	Notation usually used	Example: linear search for key x in n items
Best case	Minimum work over all inputs of size n	Ω	x is the first element $\rightarrow 1$ comparison $\rightarrow \Omega(1)$
Worst case	Maximum work over all inputs of size n	O	x is last or absent $\rightarrow n$ comparisons $\rightarrow O(n)$
Average case	Expected work, over an assumed probability distribution of inputs	Θ	on average $(n+1)/2$ comparisons $\rightarrow \Theta(n)$

The single most common student error: believing Big- O = worst case and Ω = best case. **They are unrelated concepts.** Big- O / Ω / Θ are *bounds on a function*; best/worst/average are *which function you chose to bound*. You can perfectly well say "the best-case running time of insertion sort is $O(n)$ " or "the worst-case running time of merge sort is $\Theta(n \log n)$ ". See §2.

Choosing the input size n . Usually obvious (number of elements, number of nodes). Sometimes not: for a graph it is two numbers, $|V|$ and $|E|$; for a number-theoretic algorithm on an integer N it is the **number of bits** in N , i.e. $\log N$ — which is why testing primality by trial division up to $\sqrt{N}$ is *exponential*, not polynomial, in the input size.

Likely exam questions

1. What is an algorithm? State and explain its essential properties. [5]
2. Differentiate between a priori and a posteriori analysis of algorithms. [5]
3. Distinguish between time complexity and space complexity. What is auxiliary space? [5]
4. Explain best-case, worst-case and average-case analysis with linear search as an example. [5]
5. Why is asymptotic analysis preferred to measuring actual running time? [10]

MCQ traps

- "Space complexity" vs "auxiliary space complexity" — in-place merge is asked about often; standard merge sort is $O(n)$ *auxiliary*, $O(n)$ total.
- Trial division primality is **not** polynomial time (input size is $\log N$ bits).
- Average case is **not** the mean of best and worst.

2. Asymptotic notation

Concept — in plain English first

We want to say "these two functions grow at basically the same rate" or "this one grows no faster than that one", while ignoring two things we do not care about: **constant factors** (they depend on the machine) and **small inputs** (every algorithm is fast on 5 items).

That is literally all the notation does. Every definition below has the same two escape hatches built in — a constant c to absorb the multiplier, and a threshold n_0 to ignore small inputs.

Think of them as comparison operators between growth rates:

Notation	Reads as	Rough analogy
$f = O(g)$	f grows no faster than g	$f \leq g$
$f = \Omega(g)$	f grows at least as fast as g	$f \geq g$
$f = \Theta(g)$	f grows at exactly the same rate as g	$f = g$
$f = o(g)$	f grows strictly slower than g	$f < g$
$f = \omega(g)$	f grows strictly faster than g	$f > g$

Formal definitions

Throughout, $f(n)$ and $g(n)$ map positive integers to non-negative reals.

Big-O — asymptotic upper bound

$f(n) = O(g(n))$ if there exist positive constants **c** and **n_0** such that
 $0 \leq f(n) \leq c \cdot g(n)$ for all **$n \geq n_0$** .

Beyond size n_0 , f never exceeds a constant multiple of g . This is a **guarantee / ceiling**: the algorithm will not do worse than this.

Proof example. Show $3n^2 + 5n + 100 = O(n^2)$.

For $n \geq 1$ we have $5n \leq 5n^2$ and $100 \leq 100n^2$, so

$3n^2 + 5n + 100 \leq 3n^2 + 5n^2 + 100n^2 = 108n^2$.

Take **$c = 108$** , **$n_0 = 1$** . ■

(Any valid pair works. With $n_0 = 10$, $c = 4.5$ also works: $3n^2 + 5n + 100 \leq 3n^2 + 0.5n^2 + 1n^2 = 4.5n^2$.)

Big-Omega — asymptotic lower bound

$f(n) = \Omega(g(n))$ if there exist positive constants **c** and **n_0** such that
 $0 \leq c \cdot g(n) \leq f(n)$ for all **$n \geq n_0$** .

A **floor**: the algorithm will take at least this much. Used for lower bounds on problems ("any comparison sort is $\Omega(n \log n)$ ").

Proof example. Show $3n^2 + 5n + 100 = \Omega(n^2)$. Since $5n + 100 \geq 0$ for all $n \geq 1$, $3n^2 + 5n + 100 \geq 3n^2$. Take **$c = 3$, $n_0 = 1$** . ■

Big-Theta — asymptotic tight bound

$f(n) = \Theta(g(n))$ if there exist positive constants **c_1 , c_2** and **n_0** such that **$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$** for all **$n \geq n_0$** .

Theorem (the one to quote): $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

From the two proofs above, $3n^2 + 5n + 100 = \Theta(n^2)$ with $c_1 = 3$, $c_2 = 108$, $n_0 = 1$.

Θ is the *honest* answer. O is only an upper bound, so "binary search is $O(n^2)$ " is technically true and completely useless. **Always quote the tightest bound you can.**

Little-o — strict upper bound

$f(n) = o(g(n))$ if for **every** positive constant **c** there exists **n_0** such that **$0 \leq f(n) < c \cdot g(n)$** for all **$n \geq n_0$** . Equivalently, **$\lim_{n \rightarrow \infty} f(n)/g(n) = 0$** .

The difference from Big-O in one line: **for Big-O the bound holds for *some* constant c ; for little-o it holds for *every* constant c** . Little-o means f becomes negligible relative to g .

- $2n = o(n^2)$ ✓ ($2n/n^2 = 2/n \rightarrow 0$)
- $2n^2 = o(n^2)$ ✗ ($2n^2/n^2 = 2$, not 0) — but $2n^2 = O(n^2)$ ✓

Little-omega — strict lower bound

$f(n) = \omega(g(n))$ if for every positive **c** there exists **n_0** with **$f(n) > c \cdot g(n)$** for **$n \geq n_0$** . Equivalently **$\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$** . And **$f = \omega(g) \Leftrightarrow g = o(f)$** .

Summary table — memorise this

Notation	Definition ($\exists/\forall c$)	Limit test f/g	Bound type	Analogy
$f = O(g)$	$\exists c, n_0 : f \leq cg$	finite (incl. 0)	Upper, may be loose	$\leq$
$f = o(g)$	$\forall c, \exists n_0 : f < cg$	= 0	Upper, strict	$<$
$f = \Omega(g)$	$\exists c, n_0 : f \geq cg$	> 0 (incl. ∞)	Lower, may be loose	$\geq$
$f = \omega(g)$	$\forall c, \exists n_0 : f > cg$	= ∞	Lower, strict	$>$
$f = \Theta(g)$	$\exists c_1, c_2, n_0 : c_1 g \leq f \leq c_2 g$	finite and > 0	Tight	=

Properties worth stating in an answer

Property	Statement
Reflexive	$f = O(f)$, $f = \Omega(f)$, $f = \Theta(f)$. (Not true for o and ω .)
Transitive	$f = O(g)$ and $g = O(h) \Rightarrow f = O(h)$. Same for Ω , Θ , o , ω .
Symmetric	$f = \Theta(g) \Leftrightarrow g = \Theta(f)$. Θ only.
Transpose symmetry	$f = O(g) \Leftrightarrow g = \Omega(f)$; $f = o(g) \Leftrightarrow g = \omega(f)$.
Sum rule	$O(f) + O(g) = O(\max(f, g))$. Sequential blocks: take the bigger.
Product rule	$O(f) \times O(g) = O(f \cdot g)$. Nested loops: multiply.
Constant rule	$O(c \cdot f) = O(f)$ for any constant $c > 0$.

Not a total order

Some pairs are incomparable, e.g. $f(n) = n$ and $g(n) = n^{1+\sin n}$.

Notation pedantry that gains marks: " $f(n) = O(g(n))$ " is an abuse of the equals sign. $O(g(n))$ is a *set of functions*, so the correct statement is $f(n) \in O(g(n))$. The "=" is universal convention and is one-directional: you may write $f = O(n^2)$ but never $O(n^2) = f$.

The growth-rate hierarchy — memorise the order

$O(1) < O(\log \log n) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3)$
 $< O(n^k) < O(2^n) < O(n!) < O(n^n)$

Class	Name	Typical algorithm
$O(1)$	Constant	Array index, hash lookup (avg), stack push/pop
$O(\log \log n)$	Double log	Interpolation search (avg), <code>for (i=2; i<=n; i=i*i)</code>
$O(\log n)$	Logarithmic	Binary search, balanced-BST search, <code>while (n>1) n/=2</code>
$O(\sqrt{n})$	Root	Trial division primality up to $\sqrt{n}$
$O(n)$	Linear	Linear search, one pass over an array, counting sort
$O(n \log n)$	Linearithmic	Merge sort, heap sort, quick sort (avg), FFT
$O(n^2)$	Quadratic	Bubble/selection/insertion sort, quick sort (worst)
$O(n^3)$	Cubic	Naive matrix multiply, Floyd–Warshall
$O(2^n)$	Exponential	Subset enumeration, naive Fibonacci, brute-force TSP subsets
$O(n!)$	Factorial	Permutation enumeration, brute-force TSP tours

Feel for the numbers — this is what makes the hierarchy stick:

n	log n	n log n	n^2	2^n
10	~3	~33	100	1,024
100	~7	~664	10,000	$\sim 1.3 \times 10^{30}$
1,000	~10	~9,966	10^6	astronomically large
1,000,000	~20	$\sim 2 \times 10^7$	10^{12}	—

At a billion operations per second, an $O(n^2)$ algorithm on $n = 10^6$ takes about 17 minutes; an $O(n \log n)$ algorithm takes 0.02 seconds. That is the whole argument for algorithm analysis in one line.

Likely exam questions

1. Define the Big-Oh notation. What does it represent in the context of algorithm analysis? **[5]** (*exactly the 2025 Q6*)
2. Define Big-O, Big-Omega and Big-Theta with formal definitions and one example each. **[5]**
3. Differentiate between Big-O and little-o notation. **[5]**
4. Prove that $3n^2 + 5n + 100 = \Theta(n^2)$ by finding suitable constants. **[10]**
5. Arrange the following in increasing order of growth: $n!$, 2^n , $n \log n$, $\log n$, n^2 , $\sqrt{n}$, n^3 , 1. Justify. **[10]**
6. Explain the significance of asymptotic notation. State and prove any three of its properties. **[10]**

MCQ traps

Trap	Truth
"Big-O = worst case"	No. Big-O bounds <i>any</i> chosen function. Worst case is a choice of function to bound.
" $n^2 = O(n^3)$?"	True. O is an upper bound, and a loose one is still valid.
" $n^3 = O(n^2)$?"	False.
" $2^{n+1} = O(2^n)$?"	True — $2^{n+1} = 2 \cdot 2^n$, constant factor 2.
" $2^{2n} = O(2^n)$?"	False — $2^{2n} = (2^n)^2$, not a constant multiple.
" $\log(n!) = ?$ "	$O(n \log n)$ by Stirling. A very common MCQ.
"Is $\log_2 n = O(\log_{10} n)$?"	Yes — log base change is a constant factor, so the base is dropped inside asymptotic notation.
" $n^{0.5}$ vs $\log n$ "	$\sqrt{n}$ grows faster than $\log n$.
" Θ is symmetric"	True for Θ only, not for O or Ω .

3. Time complexity of loops

Concept

There is no theory here, only a small set of patterns. Learn to recognise the pattern and read the answer off. **The rule is: count how many times the innermost statement executes, as a function of n.**

Code	Iteration count	Complexity	Why
<code>for(i=0; i<n; i++) x++;</code>	n	$O(n)$	Additive step, linear
<code>for(i=0; i<n; i+=3) x++;</code>	$n/3$	$O(n)$	Constant divisor dropped
<code>for(i=1; i<=n; i*=2) x++;</code>	$\lfloor \log_2 n \rfloor + 1$	$O(\log n)$	Multiplicative step
<code>for(i=n; i>0; i/=2) x++;</code>	$\log_2 n$	$O(\log n)$	Halving
<code>for(i=0; i*i<n; i++) x++;</code>	$\sqrt{n}$	$O(\sqrt{n})$	Runs while $i < \sqrt{n}$
<code>for(i=2; i<=n; i=i*i) x++;</code>	$\log_2 \log_2 n$	$O(\log \log n)$	Exponent doubles each time
<code>for(i=0; i<n; i++) for(j=0; j<n; j++) x++;</code>	n^2	$O(n^2)$	Independent nested loops multiply
<code>for(i=0; i<n; i++) for(j=0; j<i; j++) x++;</code>	$0+1+\dots+(n-1) = n(n-1)/2$	$O(n^2)$	Triangular sum is still quadratic
<code>for(i=0; i<n; i++) for(j=1; j<=n; j*=2) x++;</code>	$n \cdot \log n$	$O(n \log n)$	Linear $\times$ logarithmic
<code>for(i=0; i<n; i++) for(j=0; j<m; j++) x++;</code>	$n \cdot m$	$O(nm)$	Two size parameters — do not collapse to n^2
<code>for(i=1; i<=n; i++) for(j=1; j<=n; j+=i) x++;</code>	$n(1 + 1/2 + 1/3 + \dots) = n \cdot H_n$	$O(n \log n)$	Harmonic series $\approx \ln n$

<code>for(i=0;i<n;i++) x++;</code> <code>for(j=0;j<n*n;j++) y++;</code>	$n + n^2$	$O(n^2)$	Sequential: sum rule, take the max
<code>while(n>1) n = n/2;</code>	$\log_2 n$	$O(\log n)$	
<code>while(n>0) n = n - 1;</code>	n	$O(n)$	

The two rules that generate all of the above:

- **Sequential statements** → **add** (and the sum is dominated by the largest term).
- **Nested statements** → **multiply**.
- **Conditional (if/else)** → **for worst case, take the max of the two branches**.

Worked example — count exactly.

```
int count = 0;
for (i = 1; i <= n; i++)           // outer: n times
    for (j = 1; j <= n; j = j*2)    // inner: floor(log2 n) + 1 times
        count++;
```

Total executions = $n \cdot (\lfloor \log_2 n \rfloor + 1) = \Theta(n \log n)$.

Worked example — the tricky harmonic one.

```
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j = j + i)
        printf("*");
```

For a given i the inner loop runs $\lceil n/i \rceil$ times. Total = $\sum_{i=1..n} n/i = n \cdot \sum_{i=1..n} (1/i) = n \cdot H_n \approx n \cdot \ln n = \Theta(n \log n)$. Students almost always answer $\Theta(n^2)$ here.

MCQ traps

- `i *= 2` is logarithmic; `i += 2` is linear. Read the operator, not the number.
- A triangular double loop (`j < i`) is still $\Theta(n^2)$, not $\Theta(n)$.
- Two loops one after another **add** (so the answer is the larger), they do not multiply.
- `for(i=0;i<n;i++)` with `i` modified inside the body changes everything — read the body.
- `j = j + i` inside a loop over `i` gives the harmonic $\Theta(n \log n)$, not $\Theta(n^2)$.

4. Recurrence relations and the Master Theorem

Concept — in plain English first

When an algorithm calls itself, its running time is defined in terms of itself: "sorting n items costs the cost of sorting two halves, plus the cost of merging them." That self-referential equation is a **recurrence relation**. Solving it means turning it into a plain formula in n — a **closed form** — which we can then read as a Θ .

There are three tools, in increasing order of convenience:

Method	How it works	Best for
Substitution (guess-and-verify)	Guess the answer, prove it by induction	When you already suspect the answer
Recursion tree	Draw the tree of calls, sum the work per level, sum the levels	Getting a good guess; irregular splits

Master Theorem

Pattern-match into one of three cases

Divide-and-conquer of the standard shape —
use this in the exam

4.1 The substitution method

Solve $T(n) = 2T(n/2) + n$, $T(1) = 1$.*Guess:* $T(n) = O(n \log n)$, i.e. $T(n) \leq c \cdot n \log n$ for suitable c and all $n \geq n_0$.*Inductive step:* assume it holds for $n/2$, i.e. $T(n/2) \leq c \cdot (n/2) \cdot \log(n/2)$. Then

$$\begin{aligned}
T(n) &= 2T(n/2) + n \\
&\leq 2 \cdot c(n/2) \log(n/2) + n \\
&= cn (\log n - 1) + n \\
&= cn \log n - cn + n \\
&\leq cn \log n \quad \text{provided } -cn + n \leq 0, \text{ i.e. } c \geq 1
\end{aligned}$$

Base case: choose $n_0 = 2$; $T(2) = 2T(1) + 2 = 4 \leq c \cdot 2 \cdot \log 2 = 2c$ holds for $c \geq 2$.So with $c = 2$, $T(n) \leq 2n \log n$ for $n \geq 2$, hence **$T(n) = O(n \log n)$** . ■

4.2 The recursion tree method

Solve $T(n) = 2T(n/2) + n$.Draw the tree. Root does n work and has two children of size $n/2$:

Level 0:	$ \begin{array}{c} n \\ / \quad \backslash \\ n/2 \quad n/2 \\ / \quad \backslash \quad / \quad \backslash \\ n/4 \quad n/4 \quad n/4 \quad n/4 \\ \vdots \quad \vdots \\ 2^k \text{ nodes each of size } n/2^k \\ \vdots \\ n \text{ leaves each of size } 1 \end{array} $	work = n
Level 1:		work = $2 \cdot (n/2) = n$
Level 2:		work = $4 \cdot (n/4) = n$
Level k:		work = $2^k \cdot n/2^k = n$
Level log n:		work = n

- Work per level is **n at every level**.
- Number of levels: the size shrinks $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$, so there are **$\log_2 n + 1$** levels.
- Total = $n \times (\log_2 n + 1) = \mathbf{\Theta(n \log n)}$. ■

A tree with unequal work per level: $T(n) = 2T(n/2) + n^2$.Level 0: n^2 . Level 1: $2 \cdot (n/2)^2 = n^2/2$. Level 2: $4 \cdot (n/4)^2 = n^2/4$. Geometric series with ratio $1/2$:
 $n^2(1 + 1/2 + 1/4 + \dots) < 2n^2 = \mathbf{\Theta(n^2)}$. The root dominates.**A tree with an uneven split: $T(n) = T(n/3) + T(2n/3) + n$.**Every level still does n work (the pieces at each level sum to n), but the tree is lopsided. The shortest path to a leaf is $\log_3 n$ and the longest is $\log_{3/2} n$. So $n \cdot \log_3 n \leq T(n) \leq n \cdot \log_{3/2} n$, and since both are $\Theta(n \log n)$, **$T(n) = \Theta(n \log n)$** .

4.3 The Master Theorem — the exam tool

Form: $T(n) = a \cdot T(n/b) + f(n)$, where **$a \geq 1$** , **$b > 1$** are constants and $f(n)$ is asymptotically positive.Interpretation: the problem is split into **a** subproblems, each of size **n/b** , and $f(n)$ is the cost of splitting plus the cost of combining.Compute the **critical exponent** and the **watershed function**:

$e = \log_b a$ and $n^e = n^{(\log_b a)}$

Then compare $f(n)$ against n^e :

Case	Condition	Result	Reading
1	$f(n) = O(n^{(e-\epsilon)})$ for some $\epsilon > 0$ — <i>f is polynomially smaller</i>	$T(n) = \Theta(n^e)$	The leaves dominate ; the recursion does most of the work
2	$f(n) = \Theta(n^e)$ — <i>they are the same</i>	$T(n) = \Theta(n^e \cdot \log n)$	Balanced ; every level costs the same, and there are $\log n$ levels
3	$f(n) = \Omega(n^{(e+\epsilon)})$ for some $\epsilon > 0$ — <i>f is polynomially larger</i> — AND the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds for some $c < 1$ and large n	$T(n) = \Theta(f(n))$	The root dominates ; the top-level work swamps the recursion

The intuition in one sentence: compare the work done at the *leaves* ($n^{\log_b a}$ of them) with the work done at the *root* ($f(n)$); whichever is polynomially bigger wins, and if they tie, you pay that amount at each of the $\log n$ levels.

The gap. The Master Theorem does not cover every recurrence. If $f(n)$ is bigger than n^e but not *polynomially* bigger (e.g. only by a log factor), you fall into the gap between Case 2 and Case 3 and must use a recursion tree instead. Say this if you spot one — examiners reward it.

Worked examples — do all of these

(a) Merge sort: $T(n) = 2T(n/2) + n$

$a = 2, b = 2, f(n) = n. e = \log_2 2 = 1$, so $n^e = n^1 = n$.

$f(n) = n = \Theta(n^1) = \Theta(n^e) \rightarrow$ **Case 2.**

$T(n) = \Theta(n \log n)$.

(b) Binary search: $T(n) = T(n/2) + 1$

$a = 1, b = 2, f(n) = 1. e = \log_2 1 = 0$, so $n^e = n^0 = 1$.

$f(n) = 1 = \Theta(1) = \Theta(n^e) \rightarrow$ **Case 2.**

$T(n) = \Theta(\log n)$.

(c) Naive matrix multiplication (divide & conquer): $T(n) = 8T(n/2) + n^2$

$a = 8, b = 2, f(n) = n^2. e = \log_2 8 = 3$, so $n^e = n^3$.

Is $n^2 = O(n^{(3-\epsilon)})$? Yes, with $\epsilon = 1$ ($n^2 = O(n^2)$) $\rightarrow$ **Case 1.**

$T(n) = \Theta(n^3)$. (No better than the schoolbook triple loop — which is the point of Strassen.)

(d) Strassen's algorithm: $T(n) = 7T(n/2) + n^2$

$a = 7, b = 2, f(n) = n^2. e = \log_2 7 \approx 2.807$.

Is $n^2 = O(n^{(2.807-\epsilon)})$? Yes, e.g. $\epsilon = 0.3 \rightarrow$ **Case 1.**

$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$. The saving over $\Theta(n^3)$ is exactly the one multiplication saved.

(e) $T(n) = 2T(n/2) + n^2$

$a = 2, b = 2, e = \log_2 2 = 1, n^e = n$.

Is $n^2 = \Omega(n^{(1+\epsilon)})$? Yes, $\epsilon = 1$. Regularity: $a \cdot f(n/b) = 2 \cdot (n/2)^2 = n^2/2 \leq c \cdot n^2$ with $c = 1/2 < 1$ ✓.

$\rightarrow$ **Case 3. $T(n) = \Theta(n^2)$.**

(f) $T(n) = 4T(n/2) + n^2$

$e = \log_2 4 = 2, n^e = n^2. f(n) = n^2 = \Theta(n^2) \rightarrow$ **Case 2. $T(n) = \Theta(n^2 \log n)$.**

(g) $T(n) = 3T(n/4) + n \log n$

$e = \log_4 3 \approx 0.793$. Is $n \log n = \Omega(n^{(0.793+\epsilon)})$? Yes, with $\epsilon = 0.2$ ($n \log n$ grows faster than $n^{0.993}$).

Regularity: $3 \cdot (n/4) \cdot \log(n/4) \leq (3/4) \cdot n \log n$ ✓ with $c = 3/4 < 1 \rightarrow$ **Case 3.**

$T(n) = \Theta(n \log n)$.

(h) $T(n) = 9T(n/3) + n$

$e = \log_3 9 = 2, n^e = n^2. f(n) = n = O(n^{(2-\epsilon)})$ with $\epsilon = 1 \rightarrow$ **Case 1. $T(n) = \Theta(n^2)$.**

(i) A gap case: $T(n) = 2T(n/2) + n \log n$

$e = 1$, $n^e = n$. $f(n) = n \log n$ is bigger than n , but $n \log n / n^{(1+\epsilon)} \rightarrow 0$ for every $\epsilon > 0$, so it is **not polynomially** bigger. Not Case 3; not Case 2 ($n \log n \neq \Theta(n)$); not Case 1.

Master Theorem does not apply. By recursion tree, each of the $\log n$ levels costs $\Theta(n \log n)/\dots$ — carefully: level k has 2^k nodes of size $n/2^k$, each costing $(n/2^k) \log(n/2^k)$, so level cost is $n \cdot \log(n/2^k) = n(\log n - k)$. Summing $k = 0$ to $\log n$ gives $n \cdot \sum (\log n - k) = n \cdot (\log^2 n)/2 = \Theta(n \log^2 n)$.

Summary table to memorise:

Recurrence	a	b	$e = \log_b a$	$f(n)$	Case	$T(n)$
$T(n) = T(n/2) + 1$	1	2	0	1	2	$\Theta(\log n)$
$T(n) = 2T(n/2) + 1$	2	2	1	1	1	$\Theta(n)$
$T(n) = 2T(n/2) + n$	2	2	1	n	2	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n^2$	2	2	1	n^2	3	$\Theta(n^2)$
$T(n) = 4T(n/2) + n$	4	2	2	n	1	$\Theta(n^2)$
$T(n) = 4T(n/2) + n^2$	4	2	2	n^2	2	$\Theta(n^2 \log n)$
$T(n) = 4T(n/2) + n^3$	4	2	2	n^3	3	$\Theta(n^3)$
$T(n) = 7T(n/2) + n^2$	7	2	2.807	n^2	1	$\Theta(n^{2.807})$
$T(n) = 8T(n/2) + n^2$	8	2	3	n^2	1	$\Theta(n^3)$
$T(n) = 3T(n/4) + n \log n$	3	4	0.793	$n \log n$	3	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n \log n$	2	2	1	$n \log n$	gap	$\Theta(n \log^2 n)$

4.4 Subtract-and-conquer recurrences

The Master Theorem above is for **dividing** (n/b). When the subproblem shrinks by *subtraction* ($n - b$), use this separate rule:

$T(n) = a \cdot T(n - b) + f(n)$ where $f(n) = O(n^k)$:

Condition	Result
$a < 1$	$T(n) = O(n^k)$
$a = 1$	$T(n) = O(n^{(k+1)})$
$a > 1$	$T(n) = O(n^k \cdot a^{(n/b)})$ — exponential

Recurrence	Solution	Where it comes from
$T(n) = T(n-1) + 1$	$\Theta(n)$	Linear search recursively; factorial
$T(n) = T(n-1) + n$	$\Theta(n^2)$	Quick sort worst case: selection sort
$T(n) = T(n-1) + \log n$	$\Theta(n \log n)$	
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	Towers of Hanoi
$T(n) = 2T(n-1) + n$	$\Theta(2^n)$	

$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\phi^n)$, $\phi \approx 1.618$	Naive recursive Fibonacci
$T(n) = n \cdot T(n-1) + 1$	$\Theta(n!)$	Permutation generation

Likely exam questions

1. State the Master Theorem and explain its three cases. **[5]**
2. Solve $T(n) = 2T(n/2) + n$ using (a) the recursion tree method and (b) the Master Theorem. **[10]**
3. Solve the following using the Master Theorem, stating the case in each: $T(n)=4T(n/2)+n^2$, $T(n)=7T(n/2)+n^2$, $T(n)=T(n/2)+1$, $T(n)=3T(n/4)+n \log n$. **[10]**
4. What is a recurrence relation? Describe the substitution, recursion-tree and Master methods for solving recurrences, with one example each. **[15]**
5. Give a recurrence to which the Master Theorem does not apply, explain why, and solve it another way. **[10]**
6. Derive the recurrence for merge sort and quick sort (best and worst case) and solve each. **[15]**

MCQ traps

- **Case 3 needs the regularity condition** — most students forget it exists. It almost always holds for polynomial f , but you must *state* it.
- "Polynomially smaller/larger" means by a factor of n^ϵ , **not** by a log factor. That is what creates the gap.
- $T(n) = 2T(n/2) + n$ is $\Theta(n \log n)$; $T(n) = 2T(n-1) + n$ is $\Theta(2^n)$. Division vs subtraction changes everything.
- $\log_b a$ with $a = 1$ gives $e = 0$, and $n^0 = 1$ — students often mis-handle $T(n) = T(n/2) + 1$.
- Naive recursive Fibonacci is $\Theta(\phi^n) \approx \Theta(1.618^n)$, commonly (and loosely) quoted as $O(2^n)$.

5. Divide and conquer

Concept — in plain English first

The idea is the oldest one in problem solving: **a big problem is hard, so cut it into smaller copies of the same problem, solve those, and stitch the answers together**. Because the smaller problems are the *same kind* of problem, the same method applies recursively, all the way down to a base case small enough to solve outright.

Three steps, always:

Step	What happens
Divide	Break the problem into a subproblems, each of size n/b (typically 2 halves)
Conquer	Solve each subproblem recursively; if small enough, solve directly (base case)
Combine	Merge the subproblem solutions into a solution for the original

The cost is captured exactly by $T(n) = a \cdot T(n/b) + f(n)$, where $f(n)$ is the divide + combine cost — which is why §4 came first.

Where the work sits distinguishes the algorithms:

- **Merge sort** does trivial dividing and expensive combining (the merge).
- **Quick sort** does expensive dividing (the partition) and trivial combining (nothing).
- **Binary search** does trivial dividing, one subproblem, no combining.

Key points

Advantages	Naturally recursive and easy to reason about; often gives the best-known complexity; parallelises well (subproblems are independent); improves cache locality for large data
Disadvantages	Recursion overhead (stack frames); often needs extra space; poor for small inputs (hybrid algorithms switch to insertion sort below ~10 elements); recomputes overlapping subproblems if they exist — that is the gap DP fills
Applies when	Subproblems are independent (do not share sub-subproblems)

5.1 Binary search

Precondition: the array must be sorted. Compare the key with the middle element; the key can only be in one half, so discard the other. One subproblem, half the size, $O(1)$ work per step.

```
BINARY-SEARCH(A, low, high, key)
  if low > high: return NOT FOUND
  mid = low + (high - low)/2          // avoids overflow vs (low+high)/2
  if A[mid] == key : return mid
  if A[mid] > key : return BINARY-SEARCH(A, low, mid-1, key)
  else           : return BINARY-SEARCH(A, mid+1, high, key)
```

Recurrence: $T(n) = T(n/2) + \Theta(1)$, $T(1) = \Theta(1)$.

Master Theorem: $a=1$, $b=2$, $e = \log_2 1 = 0$, $f = \Theta(1) = \Theta(n^0) \rightarrow$ **Case 2** $\rightarrow T(n) = \Theta(\log n)$.

Case	Comparisons	Complexity
Best	1 (key is at mid)	$\Omega(1)$
Average	$\sim \log_2 n - 1$	$\Theta(\log n)$
Worst	$\lfloor \log_2 n \rfloor + 1$	$O(\log n)$
Space	iterative $O(1)$; recursive $O(\log n)$ stack	

Worked trace. $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$, $n = 10$, $\text{key} = 23$.

| Step | low | high | mid | A[mid] | Action |

|---|---|---|---|---|

| 1 | 0 | 9 | 4 | 16 | $16 < 23 \rightarrow$ search right, $\text{low} = 5$ |

| 2 | 5 | 9 | 7 | 56 | $56 > 23 \rightarrow$ search left, $\text{high} = 6$ |

| 3 | 5 | 6 | 5 | 23 | **Found at index 5** |

3 comparisons; $\lfloor \log_2 10 \rfloor + 1 = 4$ is the worst case.

5.2 Merge sort

Divide the array at the midpoint (trivial). **Conquer** by sorting both halves recursively.

Combine by merging two sorted arrays into one — the real work, done in $\Theta(n)$ with two pointers.

```
MERGE-SORT(A, l, r)
  if l < r:
    m = (l+r)/2
    MERGE-SORT(A, l, m)
    MERGE-SORT(A, m+1, r)
    MERGE(A, l, m, r)          //  $\Theta(n)$  using an auxiliary array
```

Recurrence: $T(n) = 2T(n/2) + \Theta(n) \rightarrow \text{Case 2} \rightarrow \Theta(n \log n)$ in all three cases (best, average, worst are identical, because the split is always at the midpoint regardless of the data).

Property	Value
Time (best / avg / worst)	$\Theta(n \log n) / \Theta(n \log n) / \Theta(n \log n)$
Auxiliary space	$\Theta(n)$ — the merge buffer
Stable?	Yes (take from the left array on ties)
In place?	No
Good for	Linked lists (merging needs no extra space there), external sorting of huge files

Number of comparisons in a merge of two runs of sizes p and q : between $\min(p,q)$ and $p+q-1$.

5.3 Quick sort

Divide by *partitioning*: choose a **pivot**, rearrange so that everything $\leq$ pivot is on its left and everything $>$ pivot on its right. The pivot is now in its final sorted position.

Conquer by recursively sorting the two sides. **Combine**: nothing to do — the array is already sorted once both sides are.

```

QUICK-SORT(A, l, r)
  if l < r:
    p = PARTITION(A, l, r)    // returns final index of pivot;  $\Theta(n)$ 
    QUICK-SORT(A, l, p-1)
    QUICK-SORT(A, p+1, r)

PARTITION(A, l, r)           // Lomuto scheme, pivot = A[r]
  pivot = A[r]; i = l-1
  for j = l to r-1:
    if A[j] <= pivot: i++; swap A[i], A[j]
  swap A[i+1], A[r]
  return i+1

```

Worked trace (Lomuto, pivot = last element). $A = [10, 80, 30, 90, 40, 50, 70]$, pivot = 70.

j	A[j]	A[j] ≤ 70?	i after	Array after
—	—	—	-1	10 80 30 90 40 50 70
0	10	yes	0	10 80 30 90 40 50 70 (swap with itself)
1	80	no	0	10 80 30 90 40 50 70
2	30	yes	1	10 30 80 90 40 50 70
3	90	no	1	10 30 80 90 40 50 70
4	40	yes	2	10 30 40 90 80 50 70
5	50	yes	3	10 30 40 50 80 90 70
final	—	—	—	swap A[4] and A[6] → 10 30 40 50 70 90 80

Pivot 70 lands at index 4. Left part [10,30,40,50] and right part [90,80] are sorted recursively.

The three cases — this is what the 2025 Q12 wanted:

Case	When	Recurrence	Complexity
Best	Pivot splits the array exactly in half every time	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Average	Pivot is random; splits are "reasonably balanced"	$T(n) \approx T(n/4) + T(3n/4) + \Theta(n)$	$\Theta(n \log n)$ with a small constant
Worst	Pivot is always the smallest or largest $\rightarrow$ split 0 and $n-1$. Happens on an already sorted or reverse-sorted array when the pivot is the first or last element	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$

Why quick sort is used anyway, despite an $O(n^2)$ worst case: its inner loop is extremely tight, it is **in-place** (only $O(\log n)$ stack), and it has excellent cache locality. Its average constant factor beats merge sort's. **Fixes for the worst case:** randomised pivot; median-of-three pivot; median-of-medians pivot (guarantees $\Theta(n \log n)$ but is slow in practice); introsort (switch to heap sort when recursion gets too deep).

Property	Value
Time (best/avg/worst)	$\Theta(n \log n)$ / $\Theta(n \log n)$ / $\Theta(n^2)$
Auxiliary space	$O(\log n)$ average recursion stack; $O(n)$ worst
Stable?	No (standard partition schemes swap non-adjacent elements)
In place?	Yes

Merge sort vs quick sort — the table to reproduce

	Merge sort	Quick sort
Where the work is	Combine (merge)	Divide (partition)
Worst case	$\Theta(n \log n)$	$\Theta(n^2)$
Average case	$\Theta(n \log n)$	$\Theta(n \log n)$, smaller constant
Space	$\Theta(n)$ auxiliary	$O(\log n)$ stack, in-place
Stable	Yes	No
Split	Always exactly half	Data-dependent
Best for	Linked lists, external sort, stability required	Arrays in memory, general-purpose

5.4 Strassen's matrix multiplication

The problem. Multiply two $n \times n$ matrices. Schoolbook: three nested loops, n^3 multiplications and $n^3 - n^2$ additions $\rightarrow \Theta(n^3)$.

Naive divide and conquer. Split each $n \times n$ matrix into four $(n/2) \times (n/2)$ blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{aligned}$$

That is **8 multiplications** of $(n/2) \times (n/2)$ matrices plus 4 additions ($\Theta(n^2)$):

$T(n) = 8T(n/2) + \Theta(n^2) \rightarrow \text{Case 1} \rightarrow \Theta(n^3)$. No gain at all.

Strassen's insight (1969). Multiplication is the expensive operation; addition and subtraction are cheap. So trade multiplications for additions. Strassen found a set of **7** products from which all four **C** blocks can be assembled:

$$\begin{aligned} P_1 &= A_{11} \cdot (B_{12} - B_{22}) \\ P_2 &= (A_{11} + A_{12}) \cdot B_{22} \\ P_3 &= (A_{21} + A_{22}) \cdot B_{11} \\ P_4 &= A_{22} \cdot (B_{21} - B_{11}) \\ P_5 &= (A_{11} + A_{22}) \cdot (B_{11} + B_{22}) \\ P_6 &= (A_{12} - A_{22}) \cdot (B_{21} + B_{22}) \\ P_7 &= (A_{11} - A_{21}) \cdot (B_{11} + B_{12}) \\ \\ C_{11} &= P_5 + P_4 - P_2 + P_6 \\ C_{12} &= P_1 + P_2 \\ C_{21} &= P_3 + P_4 \\ C_{22} &= P_5 + P_1 - P_3 - P_7 \end{aligned}$$

Recurrence: $T(n) = 7T(n/2) + \Theta(n^2)$.

Master Theorem: $e = \log_2 7 \approx 2.807$; $f(n) = n^2 = O(n^{(2.807-\epsilon)}) \rightarrow \text{Case 1}$.

$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$.

	Naive	Strassen
Multiplications per level	8	7
Additions/subtractions	4	18
Complexity	$\Theta(n^3)$	$\Theta(n^{2.807})$

Honest limitations, worth a line in the answer: the constant factor and the 18 additions make Strassen slower than the naive method for small n (crossover typically $n \approx 32\text{--}128$); it is numerically less stable; and it assumes n is a power of 2 (pad with zeros otherwise). It matters as a *theoretical* breakthrough — it proved $\Theta(n^3)$ was not optimal, and opened a line of work down to the current $\sim O(n^{2.37})$ bounds.

5.5 Other divide-and-conquer algorithms worth naming

Algorithm	Recurrence	Complexity
Finding max and min together	$T(n) = 2T(n/2) + 2$	$\Theta(n)$ with $3n/2 - 2$ comparisons (vs $2n - 2$ naively)
Binary exponentiation a^n	$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$
Closest pair of points	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Karatsuba integer multiplication	$T(n) = 3T(n/2) + \Theta(n)$	$\Theta(n^{\log_2 3}) = \Theta(n^{1.585})$
Fast Fourier Transform	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Quickselect (k-th smallest)	$T(n) = T(n/2) + \Theta(n)$ avg	$\Theta(n)$ average, $\Theta(n^2)$ worst

Tower of Hanoi	$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$, exactly $2^n - 1$ moves
----------------	----------------------	---

Likely exam questions

1. Explain the divide-and-conquer strategy. State its three steps and give two examples. [5]
2. Explain the working principle of Quick Sort. What are its average and worst-case time complexities? [10] (*exactly the 2025 Q12*)
3. Write the merge sort algorithm, derive its recurrence and solve it. Why is it always $\Theta(n \log n)$? [10]
4. Compare merge sort and quick sort on time, space, stability and practical performance. [10]
5. Explain Strassen's matrix multiplication. Derive its time complexity and compare it with the conventional method. [15]
6. Describe and compare the divide-and-conquer and greedy paradigms, with a concrete example of a problem solved by each. [15] (*exactly the 2025 Q17 — see §12 for the answer skeleton*)

MCQ traps

- Binary search requires a **sorted** array. On an unsorted array it is simply wrong, not slow.
- Merge sort is **stable**; quick sort is **not**. Heap sort is not stable either.
- Quick sort's worst case is on **already sorted** input with a first/last pivot — the opposite of what intuition suggests.
- Strassen uses **7** multiplications, not 8; the exponent is $\log_2 7 \approx 2.81$, **not** 2.71 or 2.87.
- Merge sort's auxiliary space is $\Theta(n)$, which is why heap sort ($\Theta(1)$ auxiliary, $\Theta(n \log n)$ worst) is sometimes preferred.
- Tower of Hanoi needs $2^n - 1$ moves, not 2^n .

6. The greedy method

Concept — in plain English first

A greedy algorithm builds a solution one piece at a time, and at each step it **takes whatever looks best right now, and never reconsiders**. No lookahead, no backtracking, no keeping alternatives open.

That sounds naive, and usually it *is*: for most problems the locally best choice leads you into a worse global outcome. (Take the biggest banknote first when making change with coins $\{1, 15, 25\}$ for 30 — greedy gives $25+1+1+1+1+1 = 6$ coins, optimal is $15+15 = 2$ coins.)

But for certain problems it provably works, and when it does it is by far the simplest and fastest approach — usually just "sort, then scan".

The general structure:

```
GREEDY(candidate set C)
  S = empty
  while C is not empty and S is not a complete solution:
    x = SELECT(C)           // by the greedy criterion -- the heart of the algorithm
    C = C - {x}
    if FEASIBLE(S ∪ {x}):   // does adding x keep S legal?
      S = S ∪ {x}
  if SOLUTION(S): return S else return "no solution"
```

Five components to name in an answer: **candidate set**, **selection function** (the greedy criterion), **feasibility function**, **objective function**, **solution function**.

When is greedy provably correct?

This is the part examiners look for, and the part students skip. A greedy algorithm is correct if and only if the problem has **both** of these properties:

Property	Statement	Meaning
Greedy-choice property	A globally optimal solution can be reached by making a locally optimal (greedy) choice at each step — you never have to look at the subproblem's solutions to know the first choice is safe	The first greedy choice is <i>safe</i> : some optimal solution contains it
Optimal substructure	An optimal solution to the problem contains within it optimal solutions to its subproblems	After the greedy choice, what remains is the same problem, smaller

How you actually prove it — the exchange argument. Assume an optimal solution OPT that differs from the greedy solution G. Find the first point where they differ. Show that you can *exchange* OPT's choice for greedy's choice without making OPT worse. Repeat; you transform OPT into G without ever losing value, so G is optimal too. ■

Note the shared property. Optimal substructure is required by *both* greedy and dynamic programming. What separates them is the **greedy-choice property**: greedy has it and can commit immediately; DP does not have it and must therefore evaluate all the options and choose after the fact. **That is the one-sentence answer to "greedy vs DP".**

Formal underpinning (mention it, do not develop it): the class of problems where greedy is guaranteed optimal is exactly the class of **matroids** — a set system closed under subsets and satisfying the exchange property. Prim's, Kruskal's and scheduling-to-minimise-lateness are all matroid problems.

6.1 Fractional knapsack

Problem. n items, item i has value v_i and weight w_i . Knapsack capacity W . You may take **fractions** of an item. Maximise total value.

Greedy criterion: highest value-per-unit-weight (v_i/w_i) first.

Worked example. $W = 50$; items:

Item	Value v_i	Weight w_i	Ratio v_i/w_i
1	60	10	6.0
2	100	20	5.0
3	120	30	4.0

Sort by ratio: 1, 2, 3 (already sorted).

Step	Item	Capacity left before	Taken	Value added	Capacity left after
1	1	50	all 10 kg	60	40
2	2	40	all 20 kg	100	20

6.3 Huffman coding — work this one fully

The problem. Store a file of characters using as few bits as possible. A **fixed-length** code gives every character the same number of bits, which wastes space when some characters are far more common than others. A **variable-length** code gives frequent characters short codes and rare characters long ones.

The catch: with variable-length codes, how does a decoder know where one code ends? The answer is a **prefix code** (properly, a *prefix-free* code): no codeword is a prefix of any other. Then decoding is unambiguous with no separators — read bits until they match a codeword, emit, repeat.

Huffman's algorithm builds the optimal prefix code, greedily, from the bottom up:

```
HUFFMAN(C)                                // C = set of characters with frequencies
  Q = min-priority queue of all characters, keyed by frequency
  for i = 1 to |C| - 1:
    z = new node
    z.left = x = EXTRACT-MIN(Q)
    z.right = y = EXTRACT-MIN(Q)
    z.freq = x.freq + y.freq
    INSERT(Q, z)
  return EXTRACT-MIN(Q)                    // the single remaining node is the root
```

Greedy criterion: repeatedly merge the two lowest-frequency nodes. The intuition: the two rarest symbols should sit deepest in the tree, so make them siblings at the bottom and treat their combination as a single new symbol.

Worked example

Character	a	b	c	d	e	f
Frequency	45	13	12	16	9	5

Total = 100 characters.

Trace the merges. Queue is always kept sorted by frequency.

Step	Queue before	Two smallest extracted	New node	Queue after
1	f:5, e:9, c:12, b:13, d:16, a:45	f:5, e:9	(fe):14	c:12, b:13, 14 , d:16, a:45
2	c:12, b:13, 14, d:16, a:45	c:12, b:13	(cb):25	14, d:16, 25 , a:45
3	14, d:16, 25, a:45	14, d:16	(14,d):30	25, 30 , a:45
4	25, 30, a:45	25, 30	(25,30):55	a:45, 55
5	a:45, 55	a:45, 55	root:100	root:100

The tree — draw it exactly like this. (Convention: left branch = 0, right branch = 1.)

```
      [100]
     0/   \1
    a:45   [55]
         0/   \1
        [25]   [30]
       0/  \1  0/  \1
```

```

      c:12  b:13   [14]   d:16
            0/    \1
           f:5     e:9

```

Read the codes off the tree (the path from the root):

Char	Freq	Code	Length	Freq × Length
a	45	0	1	45
b	13	101	3	39
c	12	100	3	36
d	16	111	3	48
e	9	1101	4	36
f	5	1100	4	20
	100			224 bits

Compare with a fixed-length code. Six characters need $\lceil \log_2 6 \rceil = 3$ bits each $\rightarrow 100 \times 3 = 300$ bits.

Saving = $(300 - 224)/300 = 25.3\%$.

Average codeword length = $224/100 = 2.24$ bits per character.

Verify the prefix property: 0, 101, 100, 111, 1101, 1100 — no codeword is a prefix of another.

Decode 101111100 $\rightarrow$ 101(b) 111(d) 100(c) $\rightarrow$ "bdc". Unambiguous. ✓

Complexity. Building the initial heap $\Theta(n)$; $n-1$ merges each doing two EXTRACT-MINs and one INSERT at $\Theta(\log n) \rightarrow \Theta(n \log n)$. With a pre-sorted input and two queues it drops to $\Theta(n)$.

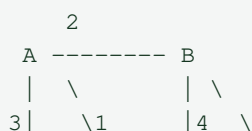
Properties to state for full marks:

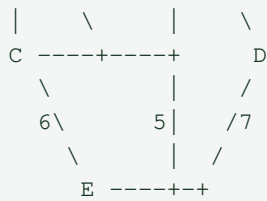
- Huffman produces an **optimal prefix code** — no other prefix code has a smaller expected length.
- It is a **full binary tree**: every internal node has exactly two children. With n characters the tree has n leaves and $n-1$ internal nodes.
- Codes are **not unique** — swapping left/right at any node gives a different but equally optimal code, and ties in the priority queue can be broken either way. **Always state your tie-breaking and 0/1 convention.**
- The most frequent character gets the shortest code; the two least frequent characters always have the **same** (maximum) length and are siblings.
- Used in DEFLATE (ZIP, gzip, PNG), JPEG and MP3 entropy coding.
- Shannon's source coding theorem gives the floor: the entropy $H = -\sum p_i \log_2 p_i$. Huffman is within 1 bit per symbol of it.

6.4 Minimum spanning tree — Prim and Kruskal

Problem. Given a connected, weighted, undirected graph, find a spanning tree ($V-1$ edges connecting all vertices, no cycle) of minimum total weight.

Worked example graph (5 vertices, 7 edges):





Edge list: A–B = 2, A–C = 3, B–C = 1, B–D = 4, C–D = 5, C–E = 6, D–E = 7.

Kruskal's algorithm — "cheapest edge anywhere, if it doesn't make a cycle"

```

KRUSKAL(G)
  sort all edges by weight, ascending
  MST = empty; make a disjoint set for each vertex
  for each edge (u,v) in sorted order:
    if FIND(u) != FIND(v):      // different components -> no cycle
      MST = MST ∪ {(u,v)}; UNION(u,v)
  if |MST| = V-1: stop
  
```

Step	Edge	Weight	Endpoints in same set?	Decision	MST so far	Total
1	B–C	1	no	accept	{BC}	1
2	A–B	2	no	accept	{BC, AB}	3
3	A–C	3	yes (A,B,C joined)	reject — cycle	{BC, AB}	3
4	B–D	4	no	accept	{BC, AB, BD}	7
5	C–D	5	yes	reject — cycle	—	7
6	C–E	6	no	accept	{BC, AB, BD, CE}	13
				4 = V-1 edges → stop		

MST = {B–C, A–B, B–D, C–E}, total weight 13.

Complexity: $\Theta(E \log E)$ for the sort ($= \Theta(E \log V)$, since $E < V^2$), plus near-constant union-find operations with path compression and union by rank. **$\Theta(E \log V)$.** Good for **sparse** graphs. Works on disconnected graphs (produces a minimum spanning *forest*).

Prim's algorithm — "grow one tree, cheapest edge leaving it"

```

PRIM(G, start)
  key[v] = ∞ for all v; key[start] = 0; parent[start] = NIL
  Q = all vertices in a min-priority queue keyed by key[]
  while Q not empty:
    u = EXTRACT-MIN(Q)
    for each neighbour v of u still in Q:
      if w(u,v) < key[v]: key[v] = w(u,v); parent[v] = u    // DECREASE-KEY
  
```

Starting at A:

Step	Tree so far	Candidate edges crossing the cut	Cheapest chosen	Total
1	{A}	A–B 2, A–C 3	A–B (2)	2

2	{A,B}	A–C 3, B–C 1 , B–D 4	B–C (1)	3
3	{A,B,C}	B–D 4, C–D 5, C–E 6	B–D (4)	7
4	{A,B,C,D}	C–E 6, D–E 7	C–E (6)	13
5	{A,B,C,D,E}	—	stop	13

MST = {A–B, B–C, B–D, C–E}, total weight 13. Same total as Kruskal — as it must be. (Here the edge sets also coincide; in general two different MSTs of equal weight can exist when weights tie. If **all edge weights are distinct, the MST is unique.**)

Complexity: with an adjacency matrix and linear scan $\Theta(V^2)$; with an adjacency list and a binary heap $\Theta(E \log V)$; with a Fibonacci heap $\Theta(E + V \log V)$. Good for **dense** graphs. Requires the graph to be connected.

Prim vs Kruskal

	Prim	Kruskal
Grows	One tree, from a start vertex outwards	A forest that merges into one tree
Selects	Cheapest edge leaving the current tree	Cheapest edge anywhere not forming a cycle
Intermediate state	Always a single connected tree	Possibly many disjoint components
Data structure	Min-priority queue on vertices	Sorted edge list + disjoint-set (union-find)
Complexity	$\Theta(V^2)$ or $\Theta(E \log V)$	$\Theta(E \log V)$
Better for	Dense graphs	Sparse graphs
Disconnected graph	Fails (needs connected)	Gives a minimum spanning forest

Correctness of both — the cut property. For any cut (partition of V into two sets), the minimum-weight edge crossing that cut belongs to some MST. Prim applies it to the cut (tree, rest); Kruskal applies it to the cut separating the two components an edge would join.

6.5 Dijkstra's shortest path

Problem. Single-source shortest paths in a weighted graph with **non-negative** edge weights.

Greedy criterion: from the unvisited vertices, always finalise the one with the smallest tentative distance. Once finalised, that distance is final — which is exactly why negative edges break it (a later negative edge could still reduce a "finalised" distance).

```

DIJKSTRA(G, s)
  dist[v] = ∞ for all v;  dist[s] = 0;  S = {}
  Q = all vertices, min-priority queue keyed by dist
  while Q not empty:
    u = EXTRACT-MIN(Q);  S = S ∪ {u}
    for each neighbour v of u:
      if dist[u] + w(u,v) < dist[v]:                // RELAXATION
        dist[v] = dist[u] + w(u,v);  prev[v] = u

```

Worked example. Directed graph, source A:

$A \rightarrow B = 4, A \rightarrow C = 2, C \rightarrow B = 1, B \rightarrow D = 5, C \rightarrow D = 8, C \rightarrow E = 10, D \rightarrow E = 2, D \rightarrow F = 6, E \rightarrow F = 3.$

Iter	Extract (min dist)	Relaxations performed	dist A	B	C	D	E	F
init	—	—	0	∞	∞	∞	∞	∞
1	A (0)	$B \leftarrow 0+4 = 4; C \leftarrow 0+2 = 2$	0	4	2	∞	∞	∞
2	C (2)	$B \leftarrow \min(4, 2+1) = 3; D \leftarrow 2+8 = 10; E \leftarrow 2+10 = 12$	0	3	2	10	12	∞
3	B (3)	$D \leftarrow \min(10, 3+5) = 8$	0	3	2	8	12	∞
4	D (8)	$E \leftarrow \min(12, 8+2) = 10; F \leftarrow 8+6 = 14$	0	3	2	8	10	14
5	E (10)	$F \leftarrow \min(14, 10+3) = 13$	0	3	2	8	10	13
6	F (13)	none	0	3	2	8	10	13

Final shortest distances from A: $B = 3, C = 2, D = 8, E = 10, F = 13.$

Shortest path to F: trace **prev** backwards — $F \leftarrow E \leftarrow D \leftarrow B \leftarrow C \leftarrow A$, i.e.

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$, length $2+1+5+2+3 = 13$. ✓

Complexity: $\Theta(V^2)$ with an array; $\Theta((V+E) \log V)$ with a binary heap; $\Theta(E + V \log V)$ with a Fibonacci heap.

Limitation — say this explicitly: Dijkstra **fails with negative edge weights**. Counterexample: $A \rightarrow B = 1, A \rightarrow C = 5, B \rightarrow C = -10$. Dijkstra finalises $B(1)$ then $C(5)$ and never revisits C , missing the true path $A \rightarrow B \rightarrow C$ of cost -9 . Use **Bellman–Ford** ($\Theta(VE)$), handles negative edges and *detects* negative cycles) or **Floyd–Warshall** (all pairs, $\Theta(V^3)$) instead.

6.6 Other greedy algorithms worth naming

Problem	Greedy criterion	Optimal?
Job sequencing with deadlines	Highest profit first, schedule as late as possible before its deadline	Yes
Minimising average completion time	Shortest job first	Yes
Coin change, canonical systems (₹1,2,5,10,20,50...)	Largest coin $\leq$ remainder	Yes for canonical systems, no in general
Graph colouring (Welsh–Powell)	Highest-degree vertex first	No — heuristic only
Bin packing (first-fit decreasing)	Largest item into the first bin that fits	No — but within $11/9 \cdot \text{OPT} + 1$
Travelling salesman (nearest neighbour)	Nearest unvisited city	No — can be arbitrarily bad
Huffman coding	Merge two lowest frequencies	Yes
Fractional knapsack	Highest value/weight	Yes
0/1 knapsack	(any)	No — needs DP

Likely exam questions

1. What is the greedy method? State its components and the two conditions under which it yields an optimal solution. **[5]**
2. Solve the fractional knapsack problem for $W = 50$ with items (60,10), (100,20), (120,30). Why does the same greedy approach fail for 0/1 knapsack? **[10]**
3. Explain Huffman coding. Construct the Huffman tree and codes for a:45, b:13, c:12, d:16, e:9, f:5, and compute the saving over a fixed-length code. **[15]**
4. Explain Prim's and Kruskal's algorithms with the same worked graph. Compare them. **[15]**
5. Explain Dijkstra's algorithm with a worked example. Why does it fail for negative edge weights? **[10]**
6. Explain the activity-selection problem. Why is "earliest finish time" the correct greedy criterion, and why does "shortest duration" fail? **[10]**

MCQ traps

Trap	Truth
"Greedy always gives the optimal solution"	No — only with the greedy-choice property + optimal substructure
"Greedy works for 0/1 knapsack"	No. Fractional yes, 0/1 no
Activity selection criterion	Earliest finish , not shortest duration and not earliest start
"Dijkstra handles negative weights"	No. Bellman–Ford does
Huffman code lengths	Not unique; only the total cost is unique
"Kruskal is better than Prim"	Depends on density — Kruskal for sparse, Prim for dense
Number of MST edges	Always $V - 1$ for a connected graph
"MST is always unique"	Only if all edge weights are distinct
Huffman tree shape	A full binary tree: n leaves, $n - 1$ internal nodes

7. Dynamic programming

Concept — in plain English first

Consider computing Fibonacci recursively: `fib(n) = fib(n-1) + fib(n-2)`. To get `fib(5)` you compute `fib(4)` and `fib(3)`. But `fib(4)` itself computes `fib(3)` — **again**. And `fib(3)` computes `fib(2)` — three separate times. The recursion tree is exponential, and almost all of it is **recomputation of identical subproblems**.

Dynamic programming's whole idea is: **compute each subproblem once, store the answer, and look it up thereafter**. That single change takes naive Fibonacci from $\Theta(1.618^n)$ to $\Theta(n)$.

This works precisely when the problem has:

Property	Meaning
----------	---------

Optimal substructure	An optimal solution is built from optimal solutions of subproblems (shared with greedy and D&C)
Overlapping subproblems	The same subproblems recur many times (the opposite of divide and conquer, where subproblems are disjoint)

Two implementation styles:

	Top-down (memoisation)	Bottom-up (tabulation)
How	Write the natural recursion; before computing, check a table; after computing, store	Fill a table iteratively, smallest subproblem first, so every dependency is ready
Control flow	Recursive	Iterative loops
Computes	Only the subproblems actually needed	All subproblems in the table
Overhead	Recursion stack + call overhead	None
Usually faster?	No	Yes , and it allows space optimisation (rolling arrays)
Easier to write?	Yes — it's just the recursion plus a cache	Requires working out the correct fill order

The four steps of designing a DP algorithm (learn these — they structure any DP answer):

1. **Characterise** the structure of an optimal solution.
2. **Recursively define** the value of an optimal solution (the recurrence).
3. **Compute** that value, typically bottom-up in a table.
4. **Construct** the optimal solution itself from the recorded information (the **traceback**).

Step 4 is worth marks and students skip it. If the question says "find the optimal solution", the table gives you the *value*; the traceback gives you the *solution*.

7.1 0/1 Knapsack — work the table

Problem. n items; item i has weight w_i and value v_i . Capacity W . Each item is taken **entirely or not at all**. Maximise value.

Why greedy fails: shown in §6.1. There is no safe local choice — whether to take an item depends on what the rest of the capacity can hold.

Recurrence. Let $K[i][w]$ = maximum value obtainable using only the first i items with capacity w .

```

K[i][w] = 0                                     if i = 0 or w = 0
K[i][w] = K[i-1][w]                             if wi > w      (item
doesn't fit)
K[i][w] = max( K[i-1][w], vi + K[i-1][w - wi] )    otherwise
               ^exclude i      ^include i

```

Worked example

$n = 4$, $W = 5$:

Item i	Weight w_i	Value v_i
----------	--------------	-------------

1	2	3
2	3	4
3	4	5
4	5	6

The table $K[i][w]$, rows $i = 0..4$, columns $w = 0..5$:

$i \setminus w$	0	1	2	3	4	5
0 (no items)	0	0	0	0	0	0
1 ($w=2, v=3$)	0	0	3	3	3	3
2 ($w=3, v=4$)	0	0	3	4	4	7
3 ($w=4, v=5$)	0	0	3	4	5	7
4 ($w=5, v=6$)	0	0	3	4	5	7

Sample cells worked out longhand — show two or three of these in an exam answer:

- $K[1][2]$: $w_1 = 2 \leq 2$, so $\max(K[0][2], 3 + K[0][0]) = \max(0, 3+0) = 3$.
- $K[2][3]$: $w_2 = 3 \leq 3$, so $\max(K[1][3], 4 + K[1][0]) = \max(3, 4+0) = 4$.
- $K[2][5]$: $w_2 = 3 \leq 5$, so $\max(K[1][5], 4 + K[1][2]) = \max(3, 4+3) = 7$.
- $K[3][4]$: $w_3 = 4 \leq 4$, so $\max(K[2][4], 5 + K[2][0]) = \max(4, 5+0) = 5$.
- $K[3][5]$: $w_3 = 4 \leq 5$, so $\max(K[2][5], 5 + K[2][1]) = \max(7, 5+0) = 7$.
- $K[4][5]$: $w_4 = 5 \leq 5$, so $\max(K[3][5], 6 + K[3][0]) = \max(7, 6+0) = 7$.

Answer: maximum value = $K[4][5] = 7$.

Traceback — which items? Start at $K[4][5]$ and walk up:

At	Compare	Conclusion	Move to
$K[4][5] = 7$	$K[3][5] = 7$ — same	item 4 not taken	$K[3][5]$
$K[3][5] = 7$	$K[2][5] = 7$ — same	item 3 not taken	$K[2][5]$
$K[2][5] = 7$	$K[1][5] = 3$ — different	item 2 taken ; $w = 5 - 3 = 2$	$K[1][2]$
$K[1][2] = 3$	$K[0][2] = 0$ — different	item 1 taken ; $w = 2 - 2 = 0$	$K[0][0]$ — stop

Optimal set = {item 1, item 2}, total weight $2 + 3 = 5$, total value $3 + 4 = 7$. ✓

Complexity. Time $\Theta(nW)$, space $\Theta(nW)$ — reducible to $\Theta(W)$ with a single rolling row iterated *right to left* (so each item is used at most once).

The pseudo-polynomial point — a guaranteed exam mark. $\Theta(nW)$ *looks* polynomial, but W is a **number**, and its size in the input is $\log W$ bits. So the running time is exponential in the input *length*. That is why 0/1 knapsack is **NP-complete** and yet has an "efficient-looking" DP: the algorithm is **pseudo-polynomial**, not polynomial. See §11.

7.2 Longest Common Subsequence — work the table

Problem. Given sequences X and Y , find the longest sequence that appears in both **as a subsequence** (in order, but not necessarily contiguous). *Subsequence $\neq$ substring*: "ACE" is a subsequence of "ABCDE" but not a substring.

Recurrence. $c[i][j]$ = length of the LCS of $X[1..i]$ and $Y[1..j]$.

```

c[i][j] = 0                                if i = 0 or j = 0
c[i][j] = c[i-1][j-1] + 1                  if xi = yj           (characters match:
extend diagonally)                          (drop one char from one
c[i][j] = max( c[i-1][j], c[i][j-1] )      if xi  $\neq$  yj           string)

```

Worked example

$X = A B C B D A B$ ($m = 7$) $Y = B D C A B A$ ($n = 6$)

	j=0	1: B	2: D	3: C	4: A	5: B	6: A
i=0	0	0	0	0	0	0	0
1: A	0	0	0	0	↖1	←1	↖1
2: B	0	↖1	←1	←1	↑1	↖2	←2
3: C	0	↑1	↑1	↖2	←2	↑2	↑2
4: B	0	↖1	←1	↑2	↑2	↖3	←3
5: D	0	↑1	↖2	↑2	↑2	↑3	↑3
6: A	0	↑1	↑2	↑2	↖3	↑3	↖4
7: B	0	↖1	↑2	↑2	↑3	↖4	←4

(↖ = characters matched, came from the diagonal; ↑ = came from above, $c[i-1][j]$; ← = came from the left, $c[i][j-1]$. Convention when $c[i-1][j] = c[i][j-1]$: take ↑.)

Sample cells worked out longhand:

- $c[1][4]$: $x_1 = A$, $y_4 = A \rightarrow$ **match** $\rightarrow c[0][3] + 1 = 0 + 1 = 1$ (↖).
- $c[2][5]$: $x_2 = B$, $y_5 = B \rightarrow$ **match** $\rightarrow c[1][4] + 1 = 1 + 1 = 2$ (↖).
- $c[3][3]$: $x_3 = C$, $y_3 = C \rightarrow$ **match** $\rightarrow c[2][2] + 1 = 1 + 1 = 2$ (↖).
- $c[4][5]$: $x_4 = B$, $y_5 = B \rightarrow$ **match** $\rightarrow c[3][4] + 1 = 2 + 1 = 3$ (↖).
- $c[5][5]$: $x_5 = D$, $y_5 = B \rightarrow$ no match $\rightarrow \max(c[4][5] = 3, c[5][4] = 2) = 3$ (↑).
- $c[7][5]$: $x_7 = B$, $y_5 = B \rightarrow$ **match** $\rightarrow c[6][4] + 1 = 3 + 1 = 4$ (↖).

LCS length = $c[7][6] = 4$.

Traceback — start at (7,6) and follow the arrows, emitting a character on every ↖:

At (i,j)	x_i, y_j	Arrow	Emit	Next
(7,6)	B, A	←	—	(6,6)
(6,6)	A, A	↖	A	(5,5)

(5,5)	D, B	↑	—	(4,5)
(4,5)	B, B	↖	B	(3,4)
(3,4)	C, A	←	—	(3,3)
(3,3)	C, C	↖	C	(2,2)
(2,2)	B, D	←	—	(2,1)
(2,1)	B, B	↖	B	(1,0) — stop

Emitted in reverse order: A, B, C, B → **LCS = "BCBA"**, length 4. ✓

(Note: "BDAB" and "BCAB" are also LCSs of length 4. The LCS is not unique; the length is.)

Complexity. Time $\Theta(mn)$, space $\Theta(mn)$; space reducible to $\Theta(\min(m,n))$ if only the *length* is needed (two rolling rows), but the traceback needs the full table — or Hirschberg's divide-and-conquer variant, which recovers the sequence in $\Theta(\min(m,n))$ space and $\Theta(mn)$ time.

Applications: `diff` and version control, DNA/protein sequence alignment (bioinformatics), plagiarism detection, spell checking (via edit distance, the close cousin of LCS).

7.3 Matrix chain multiplication

Problem. Multiply $A_1 \times A_2 \times \dots \times A_n$. Matrix multiplication is **associative**, so the answer is the same whichever way you bracket it — but the *cost* is not. Multiplying a $p \times q$ matrix by a $q \times r$ matrix costs **$p \cdot q \cdot r$** scalar multiplications. Find the parenthesisation minimising the total.

Why it matters: the difference is enormous. For $A(10 \times 100)$, $B(100 \times 5)$, $C(5 \times 50)$:

- $(AB)C = 10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 5,000 + 2,500 = \mathbf{7,500}$

- $A(BC) = 100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 25,000 + 50,000 = \mathbf{75,000}$

Ten times worse. And the number of possible parenthesisations is the Catalan number

$C(n-1) = \Omega(4^n/n^{1.5})$, so brute force is hopeless.

Recurrence. Dimensions given by $p[0..n]$, where A_i is $p[i-1] \times p[i]$.

$m[i][j]$ = minimum multiplications to compute $A_i \dots A_j$.

```

m[i][j] = 0                                     if i = j
m[i][j] = min over i ≤ k < j of { m[i][k] + m[k+1][j] + p[i-1] · p[k] · p[j] }

```

Also store $s[i][j]$ = the k achieving the minimum, for the traceback.

Worked example

$n = 4$, $p = [5, 4, 6, 2, 7]$, so $A_1 = 5 \times 4$, $A_2 = 4 \times 6$, $A_3 = 6 \times 2$, $A_4 = 2 \times 7$.

Chain length 2:

- $m[1][2] = 5 \cdot 4 \cdot 6 = \mathbf{120}$ ($k=1$)

- $m[2][3] = 4 \cdot 6 \cdot 2 = \mathbf{48}$ ($k=2$)

- $m[3][4] = 6 \cdot 2 \cdot 7 = \mathbf{84}$ ($k=3$)

Chain length 3:

- $m[1][3]: k=1 \rightarrow m[1][1] + m[2][3] + 5 \cdot 4 \cdot 2 = 0 + 48 + 40 = \mathbf{88}$

$k=2 \rightarrow m[1][2] + m[3][3] + 5 \cdot 6 \cdot 2 = 120 + 0 + 60 = 180$

→ $m[1][3] = \mathbf{88}$, $s = 1$

- $m[2][4]: k=2 \rightarrow m[2][2]+m[3][4]+4 \cdot 6 \cdot 7 = 0+84+168 = 252$

$k=3 \rightarrow m[2][3]+m[4][4]+4 \cdot 2 \cdot 7 = 48+0+56 = \mathbf{104}$

$\rightarrow m[2][4] = 104, s = 3$

Chain length 4:

- $m[1][4]: k=1 \rightarrow m[1][1]+m[2][4]+5 \cdot 4 \cdot 7 = 0+104+140 = 244$

$k=2 \rightarrow m[1][2]+m[3][4]+5 \cdot 6 \cdot 7 = 120+84+210 = 414$

$k=3 \rightarrow m[1][3]+m[4][4]+5 \cdot 2 \cdot 7 = 88+0+70 = \mathbf{158}$

$\rightarrow m[1][4] = 158, s = 3$

The m table (only the upper triangle is used):

$m[i][j]$	$j=1$	$j=2$	$j=3$	$j=4$
$i=1$	0	120	88	158
$i=2$		0	48	104
$i=3$			0	84
$i=4$				0

The s table (the optimal split point k):

$s[i][j]$	$j=2$	$j=3$	$j=4$
$i=1$	1	1	3
$i=2$		2	3
$i=3$			3

Traceback. $s[1][4] = 3 \rightarrow$ split as $(A_1A_2A_3)(A_4)$. Then $s[1][3] = 1 \rightarrow$ split as $(A_1)(A_2A_3)$.

Optimal parenthesisation: $((A_1(A_2A_3))A_4)$, **cost 158 scalar multiplications.**

Compare: the left-to-right bracketing $((A_1A_2)A_3)A_4$ costs $120 + 5 \cdot 6 \cdot 2 + 5 \cdot 2 \cdot 7 = 120+60+70 = 250$.

Complexity. $\Theta(n^3)$ time (n^2 table entries, each taking $O(n)$ to fill), $\Theta(n^2)$ space.

7.4 Floyd–Warshall — all-pairs shortest paths

The idea. Instead of asking "what is the shortest path from i to j ?", ask a smarter question:

"what is the shortest path from i to j **using only vertices $1..k$ as intermediates?**" Then grow k from 0 to n . At each step, either the new vertex k helps or it does not:

$$d^k[i][j] = \min(\underset{\text{^don't use } k}{d^{(k-1)}[i][j]}, \quad \underset{\text{^go } i \rightarrow k, \text{ then } k \rightarrow j}{d^{(k-1)}[i][k] + d^{(k-1)}[k][j]})$$

```
FLOYD-WARSHALL(W)
  D = W                                     // D[i][j] = ∞ if no edge, 0 if i = j
  for k = 1 to n:
    for i = 1 to n:
      for j = 1 to n:
        if D[i][k] + D[k][j] < D[i][j]:
          D[i][j] = D[i][k] + D[k][j]
```

```

P[i][j] = k // predecessor/via matrix for path
reconstruction

```

Note the loop order: k is the OUTERMOST loop. Putting k innermost is the classic bug and a favourite MCQ.

Worked example

Directed graph on 4 vertices: $1 \rightarrow 2 = 5$, $1 \rightarrow 4 = 10$, $2 \rightarrow 3 = 3$, $3 \rightarrow 4 = 1$.

D⁰ (the weight matrix):

	1	2	3	4
1	0	5	∞	10
2	∞	0	3	∞
3	∞	∞	0	1
4	∞	∞	∞	0

k = 1 (allow vertex 1 as intermediate): nothing enters vertex 1, so no path can route through it.

No change.

k = 2 (allow vertex 2): check $i \rightarrow 2 \rightarrow j$. $d[1][2] = 5$ and $d[2][3] = 3 \rightarrow d[1][3] = \min(\infty, 8) = 8$.

D²:

	1	2	3	4
1	0	5	8	10
2	∞	0	3	∞
3	∞	∞	0	1
4	∞	∞	∞	0

k = 3 (allow vertex 3): $d[1][3] + d[3][4] = 8 + 1 = 9 < 10 \rightarrow d[1][4] = 9$.

$d[2][3] + d[3][4] = 3 + 1 = 4 < \infty \rightarrow d[2][4] = 4$.

D³:

	1	2	3	4
1	0	5	8	9
2	∞	0	3	4
3	∞	∞	0	1
4	∞	∞	∞	0

k = 4: nothing leaves vertex 4. No change. **D⁴ = D³ is the final answer.**

Shortest path $1 \rightarrow 4$ is **9**, via $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, not the direct edge of weight 10.

Complexity. Time $\Theta(V^3)$, space $\Theta(V^2)$. Handles **negative edge weights** (unlike Dijkstra),

but not negative *cycles* — though it **detects** them: after termination, a negative entry on the diagonal ($D[i][i] < 0$) means vertex i lies on a negative cycle.

Comparison of the shortest-path algorithms:

Algorithm	Scope	Negative edges?	Complexity	Paradigm
Dijkstra	Single source	No	$\Theta((V+E) \log V)$	Greedy
Bellman–Ford	Single source	Yes (detects negative cycles)	$\Theta(VE)$	DP
Floyd–Warshall	All pairs	Yes (detects negative cycles)	$\Theta(V^3)$	DP
BFS	Single source, unweighted	n/a	$\Theta(V+E)$	Traversal

7.5 Optimal binary search tree

Problem. Given keys $k_1 < k_2 < \dots < k_n$ with search probabilities $p_1 \dots p_n$, build a BST minimising the **expected search cost** $\sum p_i \cdot (\text{depth of } k_i + 1)$. Note this is *not* the same as a balanced tree — a frequently searched key should be near the root even at the cost of imbalance.

Recurrence. $c[i][j]$ = minimum expected cost of an optimal BST on keys $k_i \dots k_j$;

$w(i,j) = p_i + \dots + p_j$.

```
c[i][i-1] = 0
c[i][j] = min over i ≤ r ≤ j of { c[i][r-1] + c[r+1][j] } + w(i,j)
```

The $+ w(i,j)$ term appears because making k_r the root pushes every key in the range one level deeper, adding $w(i,j)$ to the total.

Worked example

Keys $k_1 < k_2 < k_3$ with $p_1 = 0.5$, $p_2 = 0.3$, $p_3 = 0.2$.

Weights: $w(1,1) = 0.5$, $w(2,2) = 0.3$, $w(3,3) = 0.2$, $w(1,2) = 0.8$, $w(2,3) = 0.5$, $w(1,3) = 1.0$.

Length 1: $c[1][1] = 0.5$, $c[2][2] = 0.3$, $c[3][3] = 0.2$. Roots k_1 , k_2 , k_3 .

Length 2:

- $c[1][2]$: $r=1 \rightarrow c[1][0] + c[2][2] = 0 + 0.3 = 0.3$; $r=2 \rightarrow c[1][1] + c[3][2] = 0.5 + 0 = 0.5$.

min = 0.3, $+ w(1,2) = 0.8 \rightarrow$ **$c[1][2] = 1.1$, root = k_1**

- $c[2][3]$: $r=2 \rightarrow 0 + c[3][3] = 0.2$; $r=3 \rightarrow c[2][2] + 0 = 0.3$.

min = 0.2, $+ w(2,3) = 0.5 \rightarrow$ **$c[2][3] = 0.7$, root = k_2**

Length 3:

- $c[1][3]$: $r=1 \rightarrow c[1][0] + c[2][3] = 0 + 0.7 = 0.7$

$r=2 \rightarrow c[1][1] + c[3][3] = 0.5 + 0.2 = 0.7$

$r=3 \rightarrow c[1][2] + c[4][3] = 1.1 + 0 = 1.1$

min = 0.7, $+ w(1,3) = 1.0 \rightarrow$ **$c[1][3] = 1.7$** (root k_1 or k_2 — a tie)

Verify by hand. With root k_1 (right chain $k_1 \rightarrow k_2 \rightarrow k_3$): depths 1, 2, 3 $\rightarrow$

$0.5(1) + 0.3(2) + 0.2(3) = 0.5 + 0.6 + 0.6 = 1.7 \checkmark$

With root k_2 (balanced): depths 2, 1, 2 $\rightarrow 0.5(2) + 0.3(1) + 0.2(2) = 1.0 + 0.3 + 0.4 = 1.7 \checkmark$

Both optimal. Note that the *skewed* tree is as good as the balanced one here — precisely the point of the problem.

Complexity. $\Theta(n^3)$ with the straightforward recurrence; $\Theta(n^2)$ with Knuth's monotonicity optimisation on the root index.

7.6 Dynamic programming vs divide and conquer — the exam table

	Divide and Conquer	Dynamic Programming
Subproblems	Independent / disjoint — they do not share sub-subproblems	Overlapping — the same subproblem recurs many times
Each subproblem solved	Once, by definition (they're distinct)	Once because it is stored ; recomputation is what DP eliminates
Direction	Top-down recursion	Typically bottom-up tabulation (or top-down with memoisation)
Storage	No table; only the recursion stack	A table of subproblem results
Requires	Optimal substructure	Optimal substructure + overlapping subproblems
Recombination	Combine subproblem answers (merge, etc.)	Choose the best among subproblem answers
Typical complexity	Often $\Theta(n \log n)$	Often $\Theta(n^2)$ or $\Theta(n^3)$ — but converts exponential to polynomial
Space	Usually small	Usually large (the table)
Examples	Binary search, merge sort, quick sort, Strassen, FFT, closest pair	0/1 knapsack, LCS, matrix chain, Floyd–Warshall, Bellman–Ford, optimal BST, edit distance, Fibonacci
If applied to the other's problem	D&C on Fibonacci → exponential blow-up from recomputation	DP on merge sort → wasted table, no benefit, since no subproblem repeats

The one-line answer: *both* split a problem into subproblems, but divide and conquer's subproblems are disjoint so recursion suffices, whereas dynamic programming's subproblems overlap, so it stores results to avoid exponential recomputation.

7.7 Greedy vs dynamic programming

	Greedy	Dynamic Programming
Decision making	Makes one choice at each step and never revisits it	Considers all choices at each step and picks the best
Direction	Top-down: choose first, then solve the subproblem	Bottom-up: solve subproblems first, then choose
Requires	Optimal substructure + greedy-choice property	Optimal substructure + overlapping subproblems

Optimality	Only for problems with the greedy-choice property	Always optimal when the recurrence is correct
Speed	Faster — usually $\Theta(n \log n)$	Slower — usually $\Theta(n^2)/\Theta(n^3)$
Space	$\Theta(1)$ extra typically	Table required
The classic contrast	Fractional knapsack: $\Theta(n \log n)$	0/1 knapsack: $\Theta(nW)$

Likely exam questions

1. What is dynamic programming? State the two properties a problem must have for DP to apply. **[5]**
2. Differentiate between dynamic programming and divide and conquer, with examples of each. **[10]**
3. Differentiate between the greedy method and dynamic programming. Illustrate with the fractional and 0/1 knapsack problems. **[10]**
4. Solve the 0/1 knapsack problem for $W = 5$ and items $(w,v) = (2,3), (3,4), (4,5), (5,6)$ using dynamic programming. Show the complete table and identify the items selected. **[15]**
5. Find the longest common subsequence of $X = \text{ABCB DAB}$ and $Y = \text{BDCABA}$ using dynamic programming. Show the table and trace back the LCS. **[15]**
6. Explain matrix chain multiplication as a DP problem. Find the optimal parenthesisation for dimensions $5 \times 4, 4 \times 6, 6 \times 2, 2 \times 7$. **[15]**
7. Explain the Floyd–Warshall algorithm. Apply it to a given 4-vertex graph, showing all intermediate matrices. **[15]**
8. Distinguish between memoisation and tabulation. **[5]**

MCQ traps

Trap	Truth
"DP is always faster than recursion"	Only when subproblems overlap; otherwise the table is dead weight
Knapsack $\Theta(nW)$ is polynomial	No — pseudo-polynomial. W 's <i>encoding</i> is $\log W$ bits
Floyd–Warshall loop order	k outermost. i, j inside
"Floyd–Warshall handles negative cycles"	It detects them (negative diagonal); it does not produce meaningful distances with them
LCS vs longest common substring	LCS need not be contiguous; substring must be
"The LCS is unique"	No — only its length is
"Memoisation is bottom-up"	Memoisation is top-down ; tabulation is bottom-up
Matrix chain complexity	$\Theta(n^3)$ time, $\Theta(n^2)$ space
"Optimal BST = balanced BST"	No — probabilities, not shape, determine optimality

Bellman–Ford complexity

$O(VE)$, and it is **DP**, not greedy

8. Backtracking and branch & bound

8.1 Backtracking

Concept. Backtracking is organised, intelligent exhaustive search. You build a solution one component at a time, and **as soon as a partial solution cannot possibly be extended to a valid complete solution, you abandon it and back up** — "prune the branch". The saving comes from cutting whole subtrees of the search space without exploring them.

The search space is conceptually a **state-space tree**: the root is the empty solution; each level fixes one more component; the leaves are complete candidates. Backtracking is a **DFS of that tree with pruning by a bounding/feasibility function**.

Vocabulary to use in an answer:

Term	Meaning
State-space tree	Tree of all partial solutions
Live node	A generated node whose children have not all been explored
E-node (expanding node)	The live node currently being expanded
Dead node	A node that is not expanded further — either infeasible, or fully explored
Bounding function	The test that decides a node is dead
Promising / non-promising node	Can / cannot lead to a solution

Two formulations: *implicit constraints* (rules the components must satisfy relative to each other) and *explicit constraints* (the domain each component may take values from).

n-Queens

Problem. Place n queens on an $n \times n$ board so that no two attack each other (no shared row, column or diagonal).

Formulation. One queen per row is forced, so the solution is a vector $x[1..n]$ where $x[i]$ is the **column** of the queen in row i . That single observation collapses the search space from $C(n^2, n)$ to n^n , and pruning cuts it far further.

Feasibility test — queens in rows i and j clash if:

- $x[i] == x[j]$ — same column, **or**
- $|x[i] - x[j]| == |i - j|$ — same diagonal.

```
N-QUEENS(k, n)           // place queens in rows k..n
  for col = 1 to n:
    if PLACE(k, col):
      x[k] = col
```

```

        if k == n: print x                                // a complete solution
        else:      N-QUEENS(k+1, n)
    // no "undo" needed: x[k] is overwritten on the next iteration

```

Worked trace — 4-Queens. Columns 1–4, rows 1–4.

Step	Partial x	Action
1	$x_1 = 1$	Row 1: queen at column 1
2	$x_2 = 1?$	Same column — reject. $x_2 = 2?$ Diagonal (
3	$x_3 = 1?$ col clash. 2? diag with x_2 (	2–3
4	$x_2 = 4$	Try next column in row 2: $x_2 = 4$ ✓
5	$x_3 = 2$	1? col clash. 2 ✓ (no clash with (1,1) or (2,4))
6	$x_4 = ?$	1,2 clash by column; 3 diag with x_3 (
7	$x_3 = 3?$	col clash with $x_2=4$? no... but 3 vs $x_2=4$:
8	$x_1 = 2$	Restart row 1 at column 2
9	$x_2 = 4$	1? diag(
10	$x_3 = 1$	1 ✓
11	$x_4 = 3$	3 ✓ — SOLUTION: (2, 4, 1, 3)

The two solutions for $n = 4$ are **(2,4,1,3)** and **(3,1,4,2)** — mirror images. Board for (2,4,1,3):

```

      c1  c2  c3  c4
r1     .   Q   .   .
r2     .   .   .   Q
r3    Q   .   .   .
r4     .   .   Q   .

```

Number of solutions: $n=4 \rightarrow 2$, $n=5 \rightarrow 10$, $n=6 \rightarrow 4$, $n=8 \rightarrow 92$. Worst-case complexity $O(n!)$ — backtracking prunes heavily but does not change the exponential class.

Graph colouring (m-colouring)

Problem. Can the vertices of a graph be coloured with at most m colours so that no two adjacent vertices share a colour? (The **chromatic number** $\chi(G)$ is the smallest such m .)

```

M-COLOUR(k)                                // colour vertex k
for c = 1 to m:
    if all neighbours of k already coloured have colour ≠ c:
        x[k] = c
        if k == n: print x
    else:      M-COLOUR(k+1)

```

State-space tree has m^n leaves; complexity **$O(m^n \cdot n)$** .

Facts to know: a graph is 2-colourable iff it is **bipartite** iff it contains no odd cycle (testable in $\mathcal{O}(V+E)$ by BFS — this special case is *not* NP-hard). A complete graph K_n needs n colours. Deciding 3-colourability is **NP-complete**. **Four Colour Theorem:** every planar graph is 4-colourable. Applications: register allocation in compilers (CSD_02 §15), exam timetabling, frequency assignment, Sudoku.

Hamiltonian cycle

Problem. A cycle visiting **every vertex exactly once** and returning to the start.

Distinguish clearly from Eulerian:

	Eulerian circuit	Hamiltonian cycle
Visits every...	edge exactly once	vertex exactly once
Existence test	Easy: connected and every vertex has even degree	NP-complete — no easy criterion
Complexity	$\mathcal{O}(V+E)$ (Fleury/Hierholzer)	Exponential

Backtracking approach: build the path vertex by vertex; $x[k]$ is valid if it is adjacent to $x[k-1]$ and not already in the path; at $k = n$ also require $x[n]$ adjacent to $x[1]$. Complexity $\mathcal{O}(n!)$.

Sufficient conditions worth naming: *Dirac* — if every vertex has degree $\geq n/2$ ($n \geq 3$), the graph is Hamiltonian. *Ore* — if $\deg(u) + \deg(v) \geq n$ for every non-adjacent pair, likewise.

Other classic backtracking problems

Problem	Description
Sum of subsets	Find subsets of a set summing to a target m
Sudoku	Constraint satisfaction on a 9×9 grid
Rat in a maze / knight's tour	Path finding on a grid
Permutation generation	All orderings of n items
Crossword / word puzzles	Constraint satisfaction

8.2 Branch and bound

Concept. Backtracking is for **feasibility** problems ("find a solution satisfying the constraints"). Branch and bound is for **optimisation** problems ("find the *best* solution").

You still explore a state-space tree, but at each node you compute a **bound** — an optimistic estimate of the best value achievable in that subtree (an upper bound for maximisation, a lower bound for minimisation). You also keep the **best complete solution found so far** (the *incumbent*).

If a node's bound is no better than the incumbent, the entire subtree is pruned.

That is the only new idea: *bound + incumbent = pruning by optimality rather than by feasibility.*

Three search strategies:

Strategy	How the next E-node is chosen	Data structure	Note
FIFO branch and bound	Oldest live node first (BFS order)	Queue	Explores level by level; memory-hungry
LIFO branch and bound	Newest live node first (DFS order)	Stack	Memory-light; finds an incumbent quickly
LC (least cost) branch and bound	The live node with the best bound	Min-priority queue	Usually best in practice — goes where the promise is

Backtracking vs branch and bound

	Backtracking	Branch and bound
Problem type	Feasibility (find any/all valid solutions)	Optimisation (find the best)
Traversal	DFS only	BFS, DFS or least-cost
Pruning by	Feasibility — is this partial solution legal?	Bound — can this subtree beat the incumbent?
Stops when	A solution is found (or all found)	The whole tree is explored or pruned; optimality is proved
Extra state	The current path	Bounds + the best-so-far incumbent
Typical problems	n-Queens, graph colouring, Hamiltonian cycle, sum of subsets, Sudoku	0/1 knapsack (optimisation), TSP , job assignment, integer programming
Worst-case cost	Exponential	Exponential

Worked sketch — 0/1 knapsack by branch and bound. Order items by value/weight ratio descending.

At any node, the **upper bound** is: value taken so far, plus the *fractional* knapsack value of the remaining capacity using the remaining items. Because the fractional relaxation can never be worse than the 0/1 answer, that is a valid optimistic bound. If a node's bound $\leq$ the best complete solution found so far, prune the subtree. In practice this prunes the vast majority of the 2^n nodes.

Worked sketch — TSP by branch and bound. The lower bound at the root is $\frac{1}{2} \times \sum$ over all vertices of (the two cheapest edges incident on that vertex). At a node where some edges are already committed, adjust the bound accordingly. Prune any node whose lower bound exceeds the cheapest complete tour found so far.

Likely exam questions

1. What is backtracking? Explain the state-space tree, live node, E-node and dead node. [5]
2. Explain the n-Queens problem and solve the 4-Queens problem using backtracking, showing the state-space exploration. [10]
3. Explain the graph-colouring problem and its backtracking solution. What is the chromatic number? [10]

4. Differentiate between backtracking and branch and bound. **[5]**
5. Explain the three branch-and-bound search strategies (FIFO, LIFO, LC). **[10]**
6. Describe how branch and bound solves the 0/1 knapsack problem. How is the bound computed? **[15]**
7. Differentiate between Eulerian and Hamiltonian cycles. Explain the backtracking solution to the Hamiltonian cycle problem. **[10]**

MCQ traps

- Backtracking is **DFS** of the state-space tree; branch and bound may be BFS, DFS **or** least-cost.
- **8-Queens has 92 solutions**; 4-Queens has 2; 3-Queens and 2-Queens have **none**.
- Eulerian = every **edge**; Hamiltonian = every **vertex**. Eulerian is easy, Hamiltonian is NP-complete.
- The knapsack branch-and-bound bound uses the **fractional** relaxation — that is why it is a valid upper bound.
- Backtracking does **not** improve the worst-case complexity class; it improves the average case by pruning.

9. Search and traversal techniques

Mechanics of BFS and DFS are in [CORE_08_DataStructures.md](#). This section is the *algorithm-analysis* view: complexity, applications and how to choose.

Concept

A **traversal** visits every vertex/node in a systematic order. A **search** stops when it finds what it is looking for. Structurally they are the same algorithm; the difference is the termination condition.

The single distinguishing choice is **which frontier item to expand next**, and that is entirely determined by the data structure:

Structure	Order	Algorithm
Queue (FIFO)	Oldest first → level by level	BFS
Stack (LIFO) or recursion	Newest first → go deep	DFS
Priority queue	Cheapest first	Dijkstra, Prim, best-first, A*

BFS vs DFS

	BFS	DFS
Data structure	Queue	Stack (or recursion)
Order	Level by level from the source	As deep as possible, then backtrack

Time (adjacency list)	$\Theta(V + E)$	$\Theta(V + E)$
Time (adjacency matrix)	$\Theta(V^2)$	$\Theta(V^2)$
Space	$O(V)$ — worst case the whole frontier, i.e. $O(b^d)$ in a branching tree	$O(V)$ — the recursion depth, $O(d)$ in a tree
Finds shortest path (unweighted)?	Yes — the first time it reaches a vertex is by a minimum-edge path	No
Complete (finds a solution if one exists)?	Yes	Yes in a finite graph; no in an infinite/very deep one
Better when	Target is shallow / near the source; shortest path needed	Target is deep; memory is tight; whole-graph exploration needed
Spanning tree produced	BFS tree — shallow and wide	DFS tree — deep and narrow

Applications of BFS: shortest path in an unweighted graph; level-order traversal of a tree; testing **bipartiteness** (2-colour by alternating levels); finding connected components; peer-to-peer network flooding; web crawling; Ford–Fulkerson's Edmonds–Karp variant; GPS "nearest within k hops".

Applications of DFS: **topological sorting** of a DAG (reverse of the finish-time order); detecting **cycles** (a back edge exists $\Leftrightarrow$ there is a cycle); finding **strongly connected components** (Kosaraju's, Tarjan's); finding **articulation points and bridges**; solving mazes and puzzles; **backtracking is DFS**; path finding when any path will do.

DFS edge classification (on a directed graph, using discovery/finish times):

Edge type	When	Meaning
Tree edge	To an undiscovered vertex	Part of the DFS forest
Back edge	To an ancestor (grey vertex)	Indicates a cycle
Forward edge	To a descendant already finished	Shortcut down the tree
Cross edge	All others	Between subtrees

In an **undirected** graph only tree and back edges occur.

Tree traversals (a special case, on binary trees): **preorder** Root–Left–Right; **inorder** Left–Root–Right (yields sorted order in a BST); **postorder** Left–Right–Root (used for deletion and for evaluating expression trees); **level order** = BFS. All are $\Theta(n)$.

Likely exam questions

1. Compare BFS and DFS on data structure, complexity, space and applications. **[5]**
2. Explain BFS and DFS with a worked example graph, showing the traversal order and the resulting spanning tree. **[10]**
3. How is DFS used for topological sorting and cycle detection? Explain with an example. **[10]**

4. Why does BFS find the shortest path in an unweighted graph while DFS does not? [5]

MCQ traps

- BFS and DFS are both $\Theta(V + E)$ on an adjacency list but $\Theta(V^2)$ on an adjacency matrix. The 2025 MCQ Q31 asked exactly this.
- **Inorder** traversal of a BST yields sorted order — a guaranteed MCQ.
- Left → Right → Root is **postorder** (2025 MCQ Q32).
- BFS gives the shortest path only in **unweighted** graphs; with weights you need Dijkstra.
- Topological sort requires a **DAG**; it is undefined if there is a cycle.

10. Symbol tables

Concept

A **symbol table** is an abstract data type storing (**key, attributes**) pairs and supporting lookup by key. It is the data structure behind every name-to-meaning mapping in computing: identifiers in a compiler, labels in an assembler, entries in a data dictionary, words in a dictionary application.

The syllabus lists it under "Data Structure & Algorithm", so the expected treatment here is the **data-structure** one: what operations it must support and what to implement it with. The **compiler-specific** treatment — scope handling, what attributes are stored, when each phase reads and writes it — is in **CSD_02 §12 and §7**. A good answer mentions both.

Key points

Operations:

Operation	Meaning
<code>insert(key, attributes)</code>	Add a new entry (usually on declaration)
<code>lookup(key)</code>	Retrieve the entry for a key (on every use)
<code>delete(key)</code>	Remove an entry (usually on scope exit)
<code>update(key, attributes)</code>	Modify an existing entry
<code>set_attribute</code> / <code>get_attribute</code>	Access individual fields

Typical attributes stored (compiler context): name, type, size, scope/nesting level, storage class, memory offset or address, dimensions for arrays, parameter list and return type for functions, line number of declaration, initialisation status.

Implementations and their trade-offs — this table is the answer to the exam question:

Implementation	Search	Insert	Delete	Space	Comments
Unordered array / linear list	$O(n)$	$O(1)$ (append)	$O(n)$	$O(n)$	Simplest. Fine for very small tables. Insertion order preserved
Ordered (sorted) array	$O(\log n)$ binary search	$O(n)$ (shift)	$O(n)$	$O(n)$	Good when the table is built once and queried often
Linked list	$O(n)$	$O(1)$	$O(1)$ given the node	$O(n)$	Easy dynamic growth; no random access
Binary search tree	$O(\log n)$ avg, $O(n)$ worst	same	same	$O(n)$	Degenerates on sorted insertions
Balanced BST (AVL / red-black)	$O(\log n)$ worst	$O(\log n)$	$O(\log n)$	$O(n)$	Guaranteed bounds; supports ordered traversal
Hash table	$O(1)$ average, $O(n)$ worst	$O(1)$ avg	$O(1)$ avg	$O(n)$ + table	The standard choice in real compilers. No ordering; needs a good hash function and collision policy
Trie / digital tree	$O(L)$ where L = key length	$O(L)$	$O(L)$	Large	Independent of n ; good for prefix queries and keyword tables

Collision resolution in a hash-table symbol table: *separate chaining* (each bucket is a list — the usual choice for compilers, since it degrades gracefully) or *open addressing* (linear probing, quadratic probing, double hashing). Load factor $\alpha = n/m$; expected chain length is α .

Scope handling — the part specific to block-structured languages. A compiler for C, Pascal or Java must handle nested scopes, where an inner declaration *hides* an outer one. Two standard techniques:

1. **A stack of symbol tables** — one table per open scope. On entering a block, push a new empty table; on leaving, pop and discard it. `lookup` searches the top table, then the one below, and so on outwards to the global table. Clean and simple.
2. **One hash table with scope-level chaining** — a single table where each bucket's chain is kept in *most-recently-declared-first* order, with each entry tagged by nesting level. `lookup` naturally finds the innermost declaration first. On scope exit, remove all entries at that level (kept on a side list for efficiency).

Both give the **most-closely-nested rule** semantics: an identifier refers to the declaration in the innermost enclosing scope that declares it.

Where symbol tables are used:

Context	Contents
Compiler	Identifiers with type, scope, offset — see CSD_02
Assembler	Labels with their assigned addresses — see CSD_02 §2

Linker/loader	Global symbols for external reference resolution — CSD_02 §4–5
Debugger	Names, types and addresses, so the debugger can show source-level variables
DBMS	The data dictionary / system catalogue
Operating system	Process tables, open-file tables, page tables — the same ADT under different names

Likely exam questions

1. What is a symbol table? What operations does it support and what information does it store? **[5]**
2. Compare different implementations of a symbol table (linear list, ordered array, BST, hash table) on search, insert and delete cost. **[10]**
3. How does a compiler handle nested scopes in the symbol table? Explain with an example. **[10]**
4. Which data structure is most suitable for a compiler's symbol table, and why? **[5]**

MCQ traps

- Hash tables are $O(1)$ **average**, $O(n)$ **worst** (all keys colliding).
- A BST symbol table is $O(n)$ worst case; only a *balanced* tree guarantees $O(\log n)$.
- A **sorted array** allows $O(\log n)$ search but $O(n)$ insertion — good for static tables, bad for growing ones.
- The symbol table is used by **every** compiler phase, not just semantic analysis.

11. NP-completeness

Concept — in plain English first

Some problems we can solve fast. Sorting a million numbers takes a fraction of a second. Some problems we cannot: finding the shortest tour through 50 cities by checking every tour would outlast the universe.

The theory of NP-completeness is the mathematics that explains **why** the second kind is hard — and, crucially, it shows that thousands of these apparently unrelated hard problems are all *equivalently* hard. Crack any one of them efficiently and you crack all of them. Nobody has, in over fifty years of trying, and most computer scientists believe nobody can.

The key insight that makes the theory work is the distinction between **solving** and **checking**:

Finding a solution to a Sudoku may be hard. **Checking** a completed Sudoku is trivially easy.
 Finding a route through 50 cities shorter than 1000 km may be hard. **Checking** a proposed route is just adding up 50 numbers.

NP is exactly the class of problems where checking a proposed answer is easy.

11.1 Decision problems — the necessary preliminary

The whole theory is stated for **decision problems** — problems with a yes/no answer. This is not a restriction, just a normalisation: every optimisation problem has a decision version of essentially the same difficulty.

Optimisation version	Decision version
"Find the shortest TSP tour"	"Is there a tour of length $\leq k$?"
"Find the largest clique"	"Is there a clique of size $\geq k$?"
"Find the maximum-value knapsack packing"	"Is there a packing of value $\geq v$ within weight W ?"
"Find the chromatic number"	"Is the graph k -colourable?"

If you can solve the decision version fast, you can solve the optimisation version fast by binary searching on k . So hardness transfers between them.

11.2 The classes

P — Polynomial time

P is the set of decision problems solvable by a **deterministic** algorithm in $O(n^k)$ time for some constant k , where n is the input size.

These are the "tractable" or "efficiently solvable" problems. Examples: sorting, searching, shortest path (Dijkstra), MST (Prim/Kruskal), matrix multiplication, string matching, maximum flow, linear programming, **primality testing** (AKS, 2002 — a famous late addition), 2-colourability, Eulerian circuit.

NP — Nondeterministic Polynomial time

NP is the set of decision problems for which a proposed solution (a **certificate** or **witness**) can be **verified** in polynomial time by a deterministic algorithm.

Equivalently: the set of problems solvable in polynomial time by a **nondeterministic** Turing machine — one that can "guess" the right answer and then verify it.

Read the name correctly. NP stands for **Nondeterministic Polynomial**, not "Non-Polynomial". This is the single most common misconception, and it is a favourite MCQ.

The verification view is the one to use in an answer. For Hamiltonian cycle: the certificate is an ordering of vertices; verification is checking n edges exist and no vertex repeats — $\Theta(n)$. Verification is easy; finding the certificate is the hard part.

$P \subseteq NP$. If you can *solve* a problem in polynomial time, you can certainly *verify* a proposed answer in polynomial time — just solve it yourself and compare. The famous open question is whether the containment is strict.

NP-hard

A problem **H** is **NP-hard** if **every** problem **L** in NP reduces to H in polynomial time ($L \leq_p H$).

Informally: H is *at least as hard as* every problem in NP. **An NP-hard problem need not be in NP, and need not even be a decision problem.** The Halting Problem is NP-hard (it is undecidable, so infinitely harder). The optimisation version of TSP is NP-hard but is not a decision problem, so it is not in NP.

NP-complete

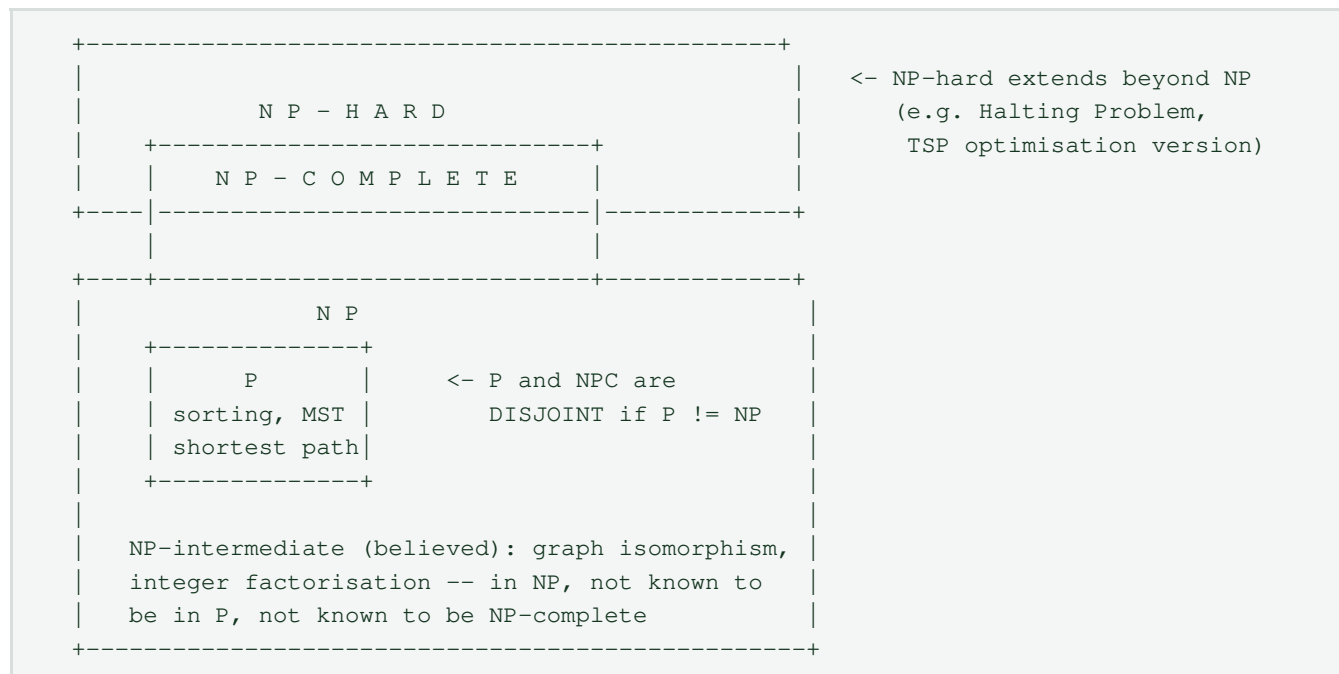
A problem is **NP-complete** if it is (1) **in NP** and (2) **NP-hard**.

NPC = NP $\cap$ NP-hard.

These are the *hardest problems in NP*. They are all equivalent in the sense that a polynomial algorithm for **any one** of them gives a polynomial algorithm for **all** of NP.

The picture — draw this diagram

If $P \neq NP$ (the believed case):



If $P = NP$: the P, NP and NP-complete circles all collapse into one. NP-hard still extends beyond.

Summary table — reproduce this:

Class	Definition	In NP?	Hardest in NP?	Examples

P	Solvable in polynomial time	Yes	No (unless $P=NP$)	Sorting, MST, shortest path, primality
NP	Verifiable in polynomial time	—	—	Everything in P, plus SAT, TSP-decision, clique...
NP-hard	Everything in NP reduces to it	Not necessarily	Yes	Halting problem, TSP-optimisation, all of NPC
NP-complete	In NP and NP-hard	Yes	Yes	SAT, 3-SAT, clique, vertex cover, Hamiltonian cycle, subset sum, graph 3-colouring

11.3 Polynomial-time reduction — the central tool

$L_1 \leq_p L_2$ (" L_1 reduces to L_2 ") means: there is a polynomial-time computable function f such that for every input x , $x \in L_1 \Leftrightarrow f(x) \in L_2$.

What it means intuitively. If you have a black box that solves L_2 , you can solve L_1 : transform your L_1 instance into an L_2 instance with f , ask the box, and the answer is the same. So

L_2 is at least as hard as L_1 .

The two directions — get these right, they are the classic exam confusion:

If you know...	And you show...	You conclude
L_1 is hard (NP-complete)	$L_1 \leq_p L_2$	L_2 is hard (NP-hard)
L_2 is easy (in P)	$L_1 \leq_p L_2$	L_1 is easy (in P)

The direction that trips people up: to prove your new problem X is NP-hard, you reduce a **known** NP-complete problem **TO** X — not X to it. You are showing " X is at least as hard as this known-hard thing", so the known-hard thing must be the *source*.

Properties: reduction is **transitive** ($L_1 \leq_p L_2$ and $L_2 \leq_p L_3 \Rightarrow L_1 \leq_p L_3$), which is exactly what lets the whole NP-complete family be built up from one root problem.

The standard four-step recipe to prove a problem X is NP-complete:

1. **Show $X \in \text{NP}$** — exhibit a certificate and a polynomial-time verifier. (Usually one or two lines. Students skip this step and lose marks.)
2. **Pick a known NP-complete problem Y .**
3. **Construct a polynomial-time reduction f from Y to X** ($Y \leq_p X$).
4. **Prove correctness:** y is a yes-instance of Y **if and only if** $f(y)$ is a yes-instance of X . Both directions.

Then: $X \in \text{NP}$ and X is NP-hard $\Rightarrow X$ is NP-complete. ■

11.4 Cook's theorem — where the chain starts

The recipe above needs a "known NP-complete problem" to start from. But how did the *first* one get proved, with nothing to reduce from?

Cook–Levin Theorem (Stephen Cook, 1971; independently Leonid Levin, 1973):
The Boolean satisfiability problem (SAT) is NP-complete.

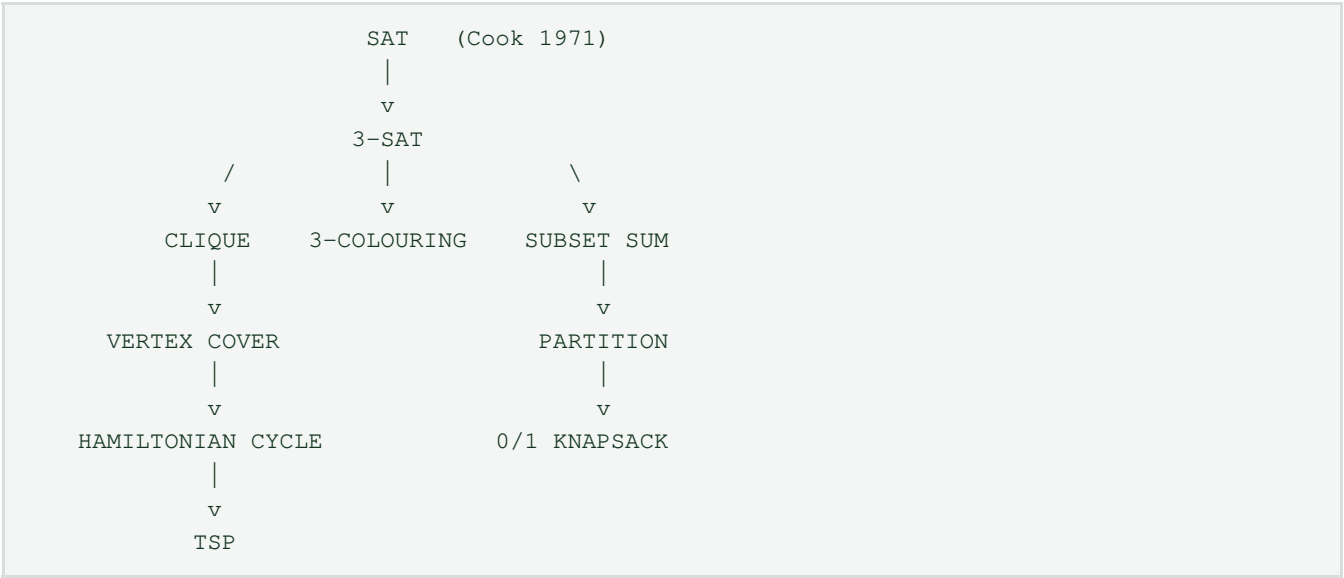
SAT: given a Boolean formula in propositional logic over variables $x_1 \dots x_n$, is there an assignment of true/false to the variables that makes the formula evaluate to true?

Example: $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$ is satisfiable ($x_1 = T, x_3 = T$). $(x \wedge \neg x)$ is not.

How Cook proved it without a predecessor. He worked from the *definition* of NP rather than from another problem. Any problem in NP has, by definition, a nondeterministic Turing machine that decides it in polynomial time. Cook showed how to build, in polynomial time, a Boolean formula that encodes the entire computation of that machine on a given input — variables representing the tape contents, head position and machine state at each of the polynomially many time steps, with clauses enforcing that consecutive configurations follow the machine's transition rules and that the final state is accepting. **The formula is satisfiable if and only if the machine accepts the input.** Since this construction works for *every* problem in NP, every problem in NP reduces to SAT. ■

Why it matters: Cook's theorem is the anchor. It gave the field its first NP-complete problem; every subsequent NP-completeness proof is a reduction chain leading back to SAT. In 1972 **Richard Karp** published "Reducibility Among Combinatorial Problems", showing **21** classic problems were NP-complete by reduction from SAT — and the floodgates opened. Thousands are known today.

The classic reduction chain — draw this:



11.5 The classic NP-complete problems — know these

Problem	Statement (decision version)
SAT	Is a given Boolean formula satisfiable? <i>(the root)</i>

3-SAT	Same, restricted to CNF with exactly 3 literals per clause. (<i>2-SAT, by contrast, is in P — a favourite trap</i>)
CLIQUE	Does G contain a complete subgraph on k vertices?
VERTEX COVER	Is there a set of k vertices touching every edge?
INDEPENDENT SET	Is there a set of k mutually non-adjacent vertices? (<i>complement of vertex cover</i>)
SUBSET SUM	Does some subset of a set of integers sum to exactly t?
PARTITION	Can a set of integers be split into two subsets of equal sum?
0/1 KNAPSACK (decision)	Is there a subset of weight $\leq W$ and value $\geq v$?
HAMILTONIAN CYCLE / PATH	Does G contain a cycle/path visiting every vertex exactly once?
TSP (decision)	Is there a tour of total cost $\leq k$?
GRAPH COLOURING ($k \geq 3$)	Is G k-colourable? (<i>2-colourability is in P</i>)
SET COVER	Can k of the given sets cover the universe?
BIN PACKING	Can the items fit into k bins of given capacity?
JOB SCHEDULING with deadlines and penalties	Various forms
SUDOKU (generalised $n^2 \times n^2$)	Is a partially filled grid completable?

Deceptive near-neighbours — the pairs examiners love:

In P (easy)	NP-complete (hard)
2-SAT	3-SAT
2-colourability (bipartiteness)	3-colourability
Eulerian circuit	Hamiltonian cycle
Shortest path	Longest simple path
Minimum spanning tree	Travelling salesman / Steiner tree
Fractional knapsack	0/1 knapsack
Matching in a bipartite graph	3-dimensional matching
Primality testing	Integer factorisation (believed hard; not known NP-complete)
Linear programming	Integer linear programming

11.6 The P vs NP question

Is $P = NP$? Is every problem whose solution can be *checked* quickly also a problem whose solution can be *found* quickly?

- One of the seven **Clay Millennium Prize Problems** — **US\$1,000,000** for a proof either way.
- Open since 1971. The overwhelming consensus is $P \neq NP$, but there is no proof.
- **To prove $P = NP$:** find a polynomial-time algorithm for any single NP-complete problem.
- **To prove $P \neq NP$:** prove that no polynomial-time algorithm exists for some problem in NP — a lower-bound proof, which is far harder to establish than an algorithm.

Consequences if $P = NP$ (this was 2025 MCQ Q34):

- Every NP-complete problem — thousands of optimisation, scheduling, routing, design and logistics problems — becomes efficiently solvable. Enormous practical gains.
- **Most public-key cryptography collapses.** RSA rests on factoring being hard; much of the rest rests on problems in NP. (Strictly: factoring is not known to be NP-complete, but $P = NP$ would place it in P as well, since it is in NP.)
- Mathematics itself changes: finding a proof of bounded length becomes as easy as checking one, so automated theorem proving becomes routine.

Consequences if $P \neq NP$: the status quo is confirmed — we must live with approximation algorithms, heuristics and exponential exact algorithms for hard problems, and cryptography's foundations are (conditionally) safe.

11.7 What to do when your problem is NP-complete

An exam answer that stops at "it's hard" is incomplete. Say what an engineer actually does:

Strategy	Description	Example
Approximation algorithms	Guarantee a solution within a proven factor of optimal in polynomial time	Vertex cover: 2-approximation by taking both endpoints of a maximal matching. Metric TSP: 1.5-approximation (Christofides)
Heuristics	No guarantee, but good in practice	Greedy, local search, simulated annealing, genetic algorithms, ant colony optimisation
Pseudo-polynomial algorithms	Polynomial in the numeric <i>value</i> of an input, not its bit length	0/1 knapsack DP in $\Theta(nW)$ — see §7.1
Parameterised / fixed-parameter tractable	Exponential only in a small parameter k , e.g. $O(2^k \cdot n)$	Vertex cover with small k
Restrict the input	The problem may be in P on a special class	3-colouring is hard in general, easy on interval graphs; many problems are polynomial on trees or planar graphs

Exact exponential with good pruning	Branch and bound, ILP solvers, SAT solvers	Modern SAT solvers routinely handle millions of variables despite the theory
Randomised algorithms	Accept a small probability of error for speed	Miller–Rabin primality

Likely exam questions

1. Define the classes P, NP, NP-hard and NP-complete. Draw a diagram showing their relationships. **[10]**
2. What does it mean for a decision problem to be in NP? Explain with an example. **[5]**
3. What is a polynomial-time reduction? How is it used to prove NP-completeness? **[10]**
4. State and explain Cook's theorem. Why is it fundamental to the theory of NP-completeness? **[10]**
5. List and briefly describe five classic NP-complete problems. **[5]**
6. What is the P vs NP problem? What would be the consequences of proving $P = NP$? **[10]**
7. Explain the concept of NP-completeness. Differentiate NP-hard from NP-complete, giving an example of a problem that is NP-hard but not NP-complete. **[15]**
8. Given that a problem is NP-complete, what practical strategies remain available? **[10]**

MCQ traps

Trap	Truth
"NP means non-polynomial"	No — Nondeterministic Polynomial. The classic error
"NP-hard problems are in NP"	No. $NP\text{-hard} \supseteq NP\text{-complete}$; NP-hard problems may be outside NP entirely
"NP-complete = NP-hard"	$NP\text{-complete} = NP\text{-hard}$ AND in NP
" $P \cap NPC = \emptyset$ "	Only if $P \neq NP$. If $P = NP$ they coincide
"The Halting Problem is NP-complete"	No — it is undecidable. It is NP-hard but not in NP
"2-SAT is NP-complete"	No, 2-SAT is in P. 3-SAT is NP-complete
"Graph 2-colouring is NP-complete"	No — it is bipartiteness testing, $O(V+E)$
"TSP is NP-complete"	The <i>decision</i> version is. The <i>optimisation</i> version is NP-hard but not in NP
"Every NP problem is NP-complete"	No — $P \subseteq NP$, and sorting is in NP but certainly not NP-complete
"Knapsack DP is polynomial so knapsack is in P"	No — pseudo-polynomial ; W is exponential in its bit length
"Factoring is NP-complete"	Not known to be. It is in NP (and co-NP), believed NP-intermediate

Which problem did Cook prove NP-complete?	SAT (Boolean satisfiability)
How many problems did Karp show NP-complete?	21

12. The master comparison table — all four paradigms

Learn this table. It is the skeleton of any "compare the algorithm design paradigms" answer, and the 2025 Section C question was exactly that.

	Divide & Conquer	Greedy	Dynamic Programming	Backtracking / B&B
Core idea	Split into independent subproblems, solve, combine	Take the locally best option, never reconsider	Solve overlapping subproblems once, store results	Systematically explore the state space, pruning dead branches
Direction	Top-down	Top-down	Bottom-up (or memoised top-down)	Top-down DFS (B&B: any order)
Subproblems	Independent	One sequence of choices	Overlapping	Tree of partial solutions
Needs optimal substructure?	For optimisation versions	Yes	Yes	Not necessarily
Needs greedy-choice property?	—	Yes	No — this is precisely why DP is needed	—
Stores intermediate results?	No	No	Yes — a table	Only the current path (+ incumbent for B&B)
Reconsiders decisions?	n/a	Never	Implicitly, by evaluating all options	Yes — by backtracking
Guarantees optimality?	Yes (it is exhaustive by construction)	Only with the greedy-choice property	Yes	Yes (it is exhaustive with pruning)
Typical complexity	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$ or $\Theta(n^3)$	Exponential
Space	$O(\log n)$ stack	$O(1)$	$O(\text{table})$	$O(\text{depth})$

Classic examples	Binary search, merge sort, quick sort, Strassen, FFT	Fractional knapsack, Huffman, Prim, Kruskal, Dijkstra, activity selection	0/1 knapsack, LCS, matrix chain, Floyd–Warshall, Bellman–Ford, optimal BST	n-Queens, graph colouring, Hamiltonian cycle, sum of subsets, TSP (B&B)
Fails when	Subproblems overlap (exponential recomputation)	The greedy-choice property does not hold (0/1 knapsack)	Subproblems don't overlap (table is wasted); or state space is too large	Search space is too large even with pruning

Answer skeleton for the 2025 Section C question (15 marks)

"Describe and compare the divide and conquer and greedy algorithmic paradigms. Provide a concrete example of a problem solved using each approach."

- 1. Introduction (2–3 lines).** Both are general strategies for designing efficient algorithms; both build a solution from smaller pieces; they differ in *how* they decompose and in whether they reconsider.
- 2. Divide and conquer.** Definition; the three steps (divide, conquer, combine); the general recurrence $T(n) = aT(n/b) + f(n)$ and the Master Theorem; the requirement that subproblems be independent.
- 3. Worked example: merge sort.** The algorithm in 4 lines; the recurrence $T(n) = 2T(n/2) + \Theta(n)$; solve to $\Theta(n \log n)$ by Master Theorem Case 2; note it is stable and needs $\Theta(n)$ auxiliary space. (*Or quick sort, with the worst-case discussion.*)
- 4. Greedy method.** Definition; the five components; the two required properties (greedy-choice + optimal substructure); the exchange-argument proof technique; the honest note that greedy fails without the greedy-choice property.
- 5. Worked example: Huffman coding.** The frequency table, the merge trace, the tree, the codes, the 224 vs 300 bit comparison. (*Or fractional knapsack, with the 240 vs greedy-fails-on-0/1 contrast.*)
- 6. Comparison table.** Four or five rows from the master table above.
- 7. Closing judgement.** When to reach for each: D&C when the problem splits into independent subproblems of the same kind; greedy when you can *prove* the local choice is safe. Add the caution that greedy is the easier and faster of the two to implement but the harder to justify, and that an unproved greedy algorithm is a bug waiting to happen — which is why 0/1 knapsack, superficially a greedy problem, requires DP.

Draw the Huffman tree and write out the merge trace. That is what turns a 10 into a 15.

Quick-reference: complexity cheat sheet

Sorting

Algorithm	Best	Average	Worst	Space	Stable	In place
Bubble	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes

Selection	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$	Yes	No
Radix	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+b)$	Yes	No
Bucket	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$	Yes	No
Shell	$\Omega(n \log n)$	depends on gap	$O(n^2)$	$O(1)$	No	Yes

Lower bound: any **comparison-based** sort requires **$\Omega(n \log n)$** comparisons in the worst case (the decision tree has $n!$ leaves, so its height is $\geq \log_2(n!) = \Theta(n \log n)$). Counting, radix and bucket sort beat this only because they are **not** comparison-based.

Searching and data structures

Operation	Array (unsorted)	Array (sorted)	Linked list	BST (avg)	BST (worst)	AVL/ RB	Hash (avg)	Heap
Search	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$	$O(n)$
Insert	$O(1)$	$O(n)$	$O(1)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$	$O(\log n)$
Delete	$O(n)$	$O(n)$	$O(1)^*$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$	$O(\log n)$
Find min	$O(n)$	$O(1)$	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(n)$	$O(1)$
(* given a pointer to the node)								

Graph algorithms

Algorithm	Complexity	Paradigm	Notes
BFS / DFS	$\Theta(V+E)$ list, $\Theta(V^2)$ matrix	Traversal	
Topological sort	$\Theta(V+E)$	DFS	DAG only
Prim (heap)	$\Theta(E \log V)$	Greedy	MST, dense $\rightarrow \Theta(V^2)$ version

Kruskal	$\Theta(E \log V)$	Greedy	MST, sparse
Dijkstra (heap)	$\Theta((V+E) \log V)$	Greedy	No negative edges
Bellman–Ford	$\Theta(VE)$	DP	Negative edges OK
Floyd–Warshall	$\Theta(V^3)$	DP	All pairs, negative edges OK
Ford–Fulkerson	$O(E \cdot \text{max-flow})$	—	Max flow
Edmonds–Karp	$O(VE^2)$	BFS-based	Max flow

Building a heap is $\Theta(n)$, not $\Theta(n \log n)$ — another perennial MCQ.

Final study instruction for this file. Do not re-read it. Take a blank sheet and reproduce, from memory: the Master Theorem's three cases; the Huffman tree for the six-character example; the 0/1 knapsack table; the LCS table and traceback; the greedy-vs-DP-vs-D&C table; and the P/NP/NP-hard/NP-complete diagram. Whatever you cannot reproduce is what you have not yet learned. Repeat until the sheet is complete. That is the whole method.